

Detección de anomalías en servicio de TV-over-IP mediante autoencoder LSTM

Ing. Christopher Charaf

Carrera de Especialización en Inteligencia Artificial

Director: Esp. Ing. María Fabiana Cid (FIUBA)

Jurados:

A designar (pertenencia)

A designar (pertenencia)

A designar (pertenencia)

Ciudad Autónoma de Buenos Aires, agosto de 2026

Resumen

En la presente memoria se describe el diseño y la implementación de un sistema de detección de anomalías para un servicio de televisión por protocolo de internet basado en interfaces de programación de aplicaciones. El trabajo fue desarrollado para una empresa proveedora de plataformas de video sobre infraestructura en la nube.

Para su concreción, fue necesaria la aplicación de aprendizaje profundo sobre series temporales mediante un autoencoder con redes neuronales recurrentes de memoria a largo plazo, junto con técnicas de ingeniería de características y de integración con sistemas de observabilidad de software como Prometheus, Grafana y Opsgenie.

Agradecimientos

A la Esp. Ing. María Fabiana Cid, directora del trabajo, por la orientación, las revisiones y el acompañamiento durante el desarrollo de esta memoria.

A la empresa Kaltura y a su equipo de soporte y operaciones, por facilitar el acceso a la infraestructura de monitoreo, por la colaboración en la calibración del umbral y por el contexto operativo que orientó el diseño del sistema.

A la Facultad de Ingeniería de la Universidad de Buenos Aires y a los docentes de la Carrera de Especialización en Inteligencia Artificial, por los conocimientos que sustentaron este proyecto.

A mi madre y a compañeros, quienes me apoyaron durante el cursado.

Índice general

Resumen	I
1. Introducción general	1
1.1. Servicios de TV-over-IP y desafíos operativos	1
1.2. Contexto y motivación	2
1.3. Estado del arte	3
1.4. Objetivos del trabajo	4
2. Introducción específica	7
2.1. Frameworks y bibliotecas de IA/ML	7
2.2. Datasets y datos de entrenamiento	8
2.2.1. Confidencialidad y entorno de demostración	8
2.2.2. Datasets de entrenamiento utilizados	9
2.3. Modelos preentrenados y transferencia de aprendizaje	9
2.4. Infraestructura de cómputo	9
2.5. Métricas de evaluación	10
3. Diseño e implementación	13
3.1. Pipeline de datos	13
3.1.1. Recolección y alineación temporal	13
3.1.2. Ingeniería de variables y normalización	15
3.1.3. Construcción de ventanas deslizantes	16
3.1.4. Organización del código fuente	17
3.1.5. Paridad entre entrenamiento e inferencia	17
3.2. Arquitectura del sistema	18
3.2.1. Encoder-Decoder LSTM	19
3.2.2. Función de pérdida y señal de anomalía	20
3.2.3. Acoplamiento multivariado y espacio latente	21
3.2.4. Decisiones de diseño arquitectónico	22
3.3. Entrenamiento, evaluación y optimización	22
3.3.1. Características del dataset	22
3.3.2. Procedimiento de entrenamiento	24
3.3.3. Calibración del umbral	25
3.3.4. Exploración de hiperparámetros	25
3.4. Integración e implementación del sistema	26
3.4.1. Servicio de inferencia	26
3.4.2. Gestión de alertas y deduplicación	27
3.4.3. Enlaces contextuales a Grafana	28
3.4.4. Despliegue y configuración	28
3.4.5. Registro, trazabilidad y requisitos no funcionales	29
3.4.6. Flujo operativo en producción	29
4. Ensayos y resultados	31

4.1. Plan de evaluación	31
4.1.1. Conjuntos de datos y partición	31
4.1.2. Confidencialidad de los resultados	32
4.1.3. Métricas y condiciones de prueba	32
4.2. Evaluación del modelo	33
4.2.1. Calidad de reconstrucción	33
4.2.2. Reconstrucción de ventanas nominales y anómalas	34
4.2.3. Desempeño de clasificación	34
4.2.4. Sensibilidad al umbral	35
4.2.5. Comparación con una línea base de umbrales fijos	36
4.2.6. Latencia de detección	36
4.3. Análisis de errores	36
4.3.1. Falsos negativos	37
4.3.2. Falsos positivos	37
4.3.3. Contribución de cada variable al error	37
4.3.4. Errores observados en el despliegue real	38
4.4. Ensayos de integración del sistema	38
4.4.1. Detección extremo a extremo	39
4.4.2. Canal de alertas intercambiable	39
4.4.3. Demostración en el entorno demostrativo	39
4.4.4. Deduplicación de alertas	41
4.4.5. Costo de inferencia y robustez	42
5. Conclusiones	45
5.1. Logros alcanzados	45
5.2. Conocimientos aplicados	46
5.3. Trabajo futuro	46
Bibliografía	49

Índice de figuras

1.1. Ejemplo conceptual de una métrica de servicio con un tramo de comportamiento anómalo. Elaboración propia.	2
3.1. Estructura lógica de ventanas deslizantes.	17
3.2. Topología de despliegue del sistema de detección.	18
3.3. Flujo interno del contenedor de detección: desde la consulta a Prometheus hasta la emisión de alertas.	19
3.4. Arquitectura del autoencoder LSTM: codificador, <i>bottleneck</i> latente y decodificador.	20
3.5. Distribución de las métricas técnicas del servicio (datos sintéticos ilustrativos).	23
4.1. Evaluación del modelo sobre el conjunto de prueba con anomalías inyectadas: distribución del error, línea de tiempo de detección, matriz de confusión y métricas.	33
4.2. Señal observada (línea continua) y reconstruida (línea discontinua) para una ventana nominal y una ventana anómala. El eje vertical está en la escala normalizada de entrada al modelo.	34
4.3. Variación de precisión, sensibilidad, F1-score y tasa de falsos positivos según el percentil del error en ventanas nominales adoptado como umbral.	35
4.4. Panel de Grafana durante la demostración en vivo: pico sostenido de latencia percentil 95 con el resto de métricas nominales.	40
4.5. Extracto de registros del detector durante la demostración: apertura, deduplicación, escalamiento y resolución.	40
4.6. Alertas en Jira Service Management Operations durante la demostración en vivo: apertura con contexto de detección y escalamiento por persistencia.	41
4.7. Ciclo de vida de una alerta: transiciones entre operación nominal, confirmación, deduplicación, escalamiento y resolución.	42

Índice de tablas

1.1. Comparación sintética de enfoques para detección de anomalías en métricas operativas (orientación conceptual).	4
2.1. Bibliotecas de terceros utilizadas en el sistema y su rol.	8
2.2. Entorno de cómputo utilizado para entrenamiento y despliegue. . .	10
3.1. Consultas PromQL y agregación por métrica del servicio.	14
3.2. Cotas de normalización <code>fixed_minmax</code> por variable.	15
3.3. Etapas del pipeline de datos y artefactos producidos.	16
3.4. Módulos implementados y responsabilidad funcional.	17
3.5. Capas del autoencoder LSTM y formas intermedias (B = tamaño de lote).	20
3.6. Asignación de prioridad Opsgenie según confianza de la anomalía. .	21
3.7. Complejidad del modelo y costo operativo estimado.	21
3.8. Descripción del conjunto de datos utilizado para entrenamiento y evaluación.	23
3.9. Hiperparámetros de entrenamiento del autoencoder LSTM.	24
3.10. Artefactos del modelo y del pipeline utilizados en inferencia. . . .	25
3.11. Exploración de configuraciones alternativas (<code>tune_model.py</code>). . .	26
3.12. Archivos de configuración YAML del sistema.	28
3.13. Comparación entre entorno demo y despliegue productivo.	29
4.1. Anomalías inyectadas en el conjunto de prueba de 24 horas.	32
4.2. Métricas de clasificación con el umbral desplegado (percentil 99,5 de validación).	35
4.3. Barrido del umbral sobre percentiles del error en ventanas nominales. .	35
4.4. Comparación entre la línea base de umbrales fijos y el autoencoder LSTM.	36
4.5. Latencia de detección por tipo de anomalía inyectada.	37
4.6. Error de reconstrucción por variable técnica: mediana en ventanas nominales y media en ventanas anómalas.	38
4.7. Escenarios de integración ejercitados en el entorno demostrativo. .	39
4.8. Efecto de la deduplicación sobre un incidente sostenido.	42

Capítulo 1

Introducción general

En el presente capítulo se contextualiza la detección de anomalías en servicios de televisión por protocolo de internet (TV-over-IP) [1]. Además, se revisa el estado del arte y se enuncian los objetivos del trabajo.

1.1. Servicios de TV-over-IP y desafíos operativos

Los servicios de video bajo demanda y de transmisión lineal sobre redes IP consolidaron en la última década un modelo en que la calidad percibida por el usuario depende tanto de la disponibilidad de los componentes de *backend* como del desempeño de la cadena completa: balanceadores, microservicios, bases de datos, colas de mensajes y sistemas de caché, entre otros. La operación de estos sistemas se apoya de manera casi universal en la recolección periódica de métricas técnicas (latencias, tasas de error, uso de CPU o memoria, etc.) que describen el estado del servicio en el tiempo.

En un servicio de TV-over-IP, el contenido audiovisual se origina en sistemas de producción o almacenamiento [1], se distribuye a través de redes de entrega y se expone al usuario final mediante aplicaciones o decodificadores físicos. Cualquier eslabón de esa cadena puede degradar la experiencia de visualización aunque el resto del sistema siga respondiendo.

Un origen saturado, un cuello de botella en la entrega o una API lenta se manifiestan para el espectador como interrupciones, demoras en el arranque de la reproducción o pérdida de calidad percibida.

Desde el punto de vista operativo, las degradaciones relevantes suelen adoptar formas reconocibles: incrementos sostenidos de latencia, picos en la tasa de errores, caídas abruptas de la capacidad de atención simultánea o combinaciones de esas señales que no coinciden con un único síntoma aislado.

El *machine learning* aporta métodos que aprenden, a partir de datos históricos, qué aspecto tiene el comportamiento nominal sin fijar a priori todas las reglas. En particular, las redes neuronales recurrentes con unidades de memoria a largo plazo (LSTM, por sus siglas en inglés) permiten modelar dependencias temporales en series multivariadas [2]. Los *autoencoders* basados en LSTM pueden entrenarse para reconstruir ventanas recientes de métricas. Cuando la reconstrucción falla de manera sistemática, el error sirve como señal de anomalía [3]. Este tipo de enfoque es comprensible para un lector familiarizado con monitoreo de infraestructura aunque no especializado en aprendizaje profundo: la idea central es que

el modelo aprende un patrón habitual y marca desviaciones fuertes respecto de ese patrón.

Una anomalía en este contexto operativo puede entenderse como un patrón de comportamiento en esas señales que se aparta de lo observado cuando el sistema funciona de forma nominal, ya sea por degradación, por un incidente puntual o por una combinación de factores que los umbrales convencionales no caracterizan bien.

La figura 1.1 ilustra esta idea sobre una métrica sintética observada a lo largo de un día: la banda verde representa el régimen nominal esperado para esa hora y la curva azul, las observaciones reales. El tramo rojo marca una ventana en la que el comportamiento se aparta del patrón habitual y representa el tipo de desvío sostenido que se busca identificar de manera automática. Detectar esas situaciones de manera temprana importa porque reduce el tiempo de detección de incidentes y acota el impacto en la experiencia de visualización y en los acuerdos de nivel de servicio.

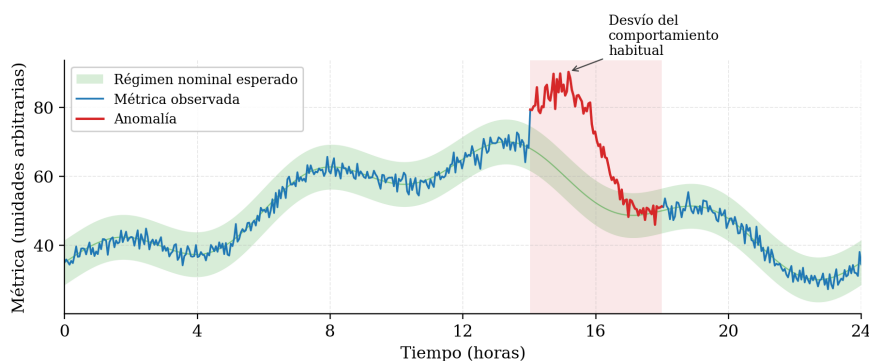


FIGURA 1.1. Ejemplo conceptual de una métrica de servicio con un tramo de comportamiento anómalo. Elaboración propia.

1.2. Contexto y motivación

La motivación del trabajo final surge de una aplicación real en el sector de plataformas de video empresarial: la organización de referencia provee servicios de TV-over-IP sobre infraestructura en la nube y requirió modernizar la forma en que se detectan fallos y degradaciones en producción. Las métricas del servicio se concentran en herramientas estándar de la industria, en particular en un sistema de acumulación de series temporales del tipo de Prometheus [4, 5], con muestreo en el orden de decenas de segundos.

Las prácticas tradicionales de alertas sobre umbrales fijos o reglas manuales siguen siendo útiles para síntomas claros, pero muestran debilidades frente a patrones multivariados, estacionalidades (por ejemplo variaciones por horario comercial o fines de semana) y dependencias que no se traducen en un único umbral intuitivo. Además, externalizar el análisis de métricas a terceros puede no ser deseable desde el punto de vista de privacidad y superficie de ataque.

En el entorno de la organización de referencia se acumularon reglas de alerta sobre muchas series en paralelo. Con el tiempo, una fracción creciente de notificaciones correspondió a situaciones que no requerían intervención inmediata o que repetían patrones estacionales ya conocidos.

Esa carga incrementó el esfuerzo del equipo de operaciones en la priorización de incidentes y elevó el riesgo de desatender eventos realmente críticos. Los falsos positivos repetidos también afectaron la confianza en el canal de alertas y dificultaron sostener acuerdos de nivel de servicio cuando las degradaciones no se detectaban con la anticipación deseada.

Por estas razones se buscó una detección más contextual, capaz de considerar varias señales a la vez, y se priorizó mantener la telemetría y el entrenamiento del modelo dentro del perímetro controlado por la empresa, sin depender de servicios analíticos externos para el núcleo de la detección.

Por ello se planteó un sistema integrado al monitoreo existente, orientado a procesar ventanas multivariadas y a emitir alertas contextualizadas cuando el patrón observado se aparta del régimen nominal aprendido. La integración con herramientas de observabilidad ya adoptadas permitió reutilizar la recolección y los tableros existentes. El esfuerzo de este trabajo se concentró en el núcleo analítico y en el encadenamiento hacia sistemas de respuesta a incidentes.

1.3. Estado del arte

La detección de anomalías abarca desde métodos estadísticos y basados en distancia hasta técnicas de aprendizaje profundo adaptadas a datos secuenciales. Existe una vasta literatura de marcos de referencia y taxonomías [6].

En operaciones de software, los datos son en la práctica series multivariadas en las que distintas métricas capturan facetas acopladas del sistema, lo que motiva modelos que traten la ventana temporal completa en lugar de decidir variable por variable de forma independiente, para capturar no linealidades y dependencias largas. En monitorización de aplicaciones e ingeniería de confiabilidad de sitios, la práctica habitual combina métricas de utilización, saturación y errores con umbrales declarativos. Ese enfoque resulta adecuado para síntomas univariados claros, pero pierde expresividad cuando un incidente modula en forma acoplada varias series antes de que cualquiera de ellas supere un límite fijo [4].

Los métodos clásicos (controles por umbrales, suavizados exponenciales o *isolation forest* [7] sobre vectores de características) conservan ventajas de simplicidad e interpretabilidad, pero pueden exigir mucho ajuste manual cuando el número de series crece o cuando la normalidad cambia lentamente en el tiempo. Los autoencoders y las arquitecturas recurrentes ofrecen una capa adicional de expresividad para capturar no linealidades y dependencias largas.

Entre los enfoques de aprendizaje profundo para series temporales, Malhotra et al. [3] entrenaron redes codificadoras-decodificadoras LSTM exclusivamente con ventanas consideradas normales y utilizaron el error de reconstrucción como indicador de anomalía. Esa línea resultó aplicable al monitoreo operativo multivariado, aunque con variables y régimen de muestreo propios del dominio de *streaming* empresarial. Trabajos posteriores exploraron arquitecturas con atención, convoluciones temporales o modelos generativos. Para telemetría con pocas decenas de variables y muestreo periódico fijo, un autoencoder LSTM compacto sigue siendo una base interpretable y eficiente en CPU.

Paralelamente, los ecosistemas de observabilidad comercial integran funciones de detección asistida o modelos propietarios sobre telemetría ya ingestada. Esas

soluciones priorizan la integración con la plataforma, pero suelen ofrecer poco control sobre el entrenamiento, el umbral y la auditoría del modelo.

La propuesta de este trabajo se acopla al *stack* Prometheus–Grafana–Opsgenie [8, 9, 10] ya adoptado en la organización. Así se busca un equilibrio entre sensibilidad y falsos positivos calibrable con el equipo de operaciones.

La tabla 1.1 resume familias de enfoques y su relación típica con datos y métricas de evaluación.

Para fines orientativos, Malhotra et al. [3] entrenaron un encoder-decoder LSTM sobre series multivariadas de sensores y utilizaron el error de reconstrucción como señal de anomalía; Liu et al. [7] aplicaron *isolation forest* sobre vectores de características con puntaje de anomalía; en esta memoria se adoptó un autoencoder LSTM sobre telemetría operativa de TV-over-IP con error de reconstrucción y umbral calibrado. Ninguno de estos resúmenes pretende ser exhaustivo, sino ubicar la propuesta en el panorama habitual de ingeniería de confiabilidad y aprendizaje automático.

TABLA 1.1. Comparación sintética de enfoques para detección de anomalías en métricas operativas (orientación conceptual).

Enfoque	Tipo de modelo o regla	Datos típicos	Métricas habituales
Umbrales fijos	Reglas fijas	Series univariadas	Tiempo de detección alto, fasas alertas
Modelos clásicos	kNN, Isolation Forest, etc.	Ventanas vectorizadas	Precisión, <i>recall</i> , F1
Modelos secuenciales profundos	Autoencoder LSTM, RNN	Ventanas multivariadas	Error de reconstrucción, F1
Productos comerciales	Algoritmos propietarios	Telemetría agregada	Integración con el proveedor

1.4. Objetivos del trabajo

El objetivo general del trabajo consistió en diseñar e implementar un sistema inteligente capaz de detectar anomalías en tiempo de operación, compatible con el monitoreo existente en un servicio de TV-over-IP basado en API, aprovechando la integración con sistemas de monitoreo y alertas para mejorar la capacidad de reacción frente a incidentes. Esto permite, por otra parte, evitar la dependencia de umbrales estáticos o problemas de seguridad al utilizar servicios de terceros.

Los objetivos específicos, formulados de manera verificable, fueron los siguientes:

1. Identificar y preparar las métricas críticas del servicio disponibles vía API de Prometheus, incluyendo normalización, variables de contexto temporal (por ejemplo codificación cíclica de la hora) y construcción de ventanas deslizantes acordes al período de muestreo del entorno.
2. Entrenar y ajustar un modelo autoencoder LSTM capaz de reconstruir el comportamiento nominal del sistema y traducir el error de reconstrucción en una señal de anomalía gobernada por un umbral calibrado con el equipo de operaciones.

3. Integrar el flujo de inferencia con mecanismos de alerta y la generación de enlaces a paneles en Grafana que permitan inspeccionar el intervalo temporal asociado a cada anomalía detectada.
4. Evaluar el sistema sobre datos históricos representativos del régimen normal de operación, reportando al menos la tasa de falsos positivos y la capacidad de detección en escenarios acordados con las partes interesadas, de forma que los resultados en esta memoria permitan contrastar el desempeño frente a líneas base definidas en el capítulo de ensayos.

Capítulo 2

Introducción específica

En el presente capítulo se describen los recursos de software, los conjuntos de datos y la infraestructura de cómputo de terceros que sustentan el sistema documentado en esta memoria. Se justifica la elección de cada componente y se caracteriza el origen de la telemetría utilizada para el entrenamiento.

2.1. Frameworks y bibliotecas de IA/ML

La implementación del autoencoder LSTM se apoyó en `TensorFlow` y su interfaz `Keras` [11, 12], que ofrecen una capa de alto nivel para definir, entrenar y serializar redes neuronales profundas. Entre las capas utilizadas figuran `LSTM`, `RepeatVector` y `TimeDistributed`, componentes nativos que permiten expresar la arquitectura codificador–decodificador con pocas líneas. La elección del *framework* obedeció a la integración estable con el ecosistema Python usado para procesamiento de datos, a la portabilidad del modelo entrenado a contenedores en CPU y al formato de serialización `.weights.h5`, que se acompaña de un archivo JSON con la definición estructural del grafo para reconstruir el modelo en inferencia sin acoplarse a una versión particular del entorno de entrenamiento.

Para el preprocesamiento se utilizó `scikit-learn` [13]. La normalización se realizó con `MinMaxScaler`, configurado con cotas fijas para desacoplar la transformación de la distribución particular de los datos de entrenamiento. La persistencia del preprocesador, necesaria para garantizar la misma transformación entre entrenamiento e inferencia, se gestionó con `joblib` [14], biblioteca de serialización binaria habitual en ese ecosistema.

La manipulación de las series temporales se apoyó en `NumPy` [15] y `pandas` [16], estándares en Python para arreglos n-dimensionales y tablas indexadas por tiempo. La obtención de métricas desde Prometheus se realizó vía HTTP con la biblioteca `requests` [17], mientras que la instrumentación del servicio simulado utilizó el cliente oficial `prometheus_client` [18], que expone contadores e histogramas en el formato esperado por el recolector. La carga de archivos de configuración en formato YAML se delegó en `PyYAML` [19], y la generación de gráficos de evaluación *offline* se realizó con `matplotlib` y `seaborn` [20, 21].

En entornos de observabilidad, la elección del stack analítico no es independiente del stack de despliegue: bibliotecas con dependencias pesadas o con requisitos de GPU dificultan la ejecución periódica en contenedores pequeños. Se priorizaron componentes maduros en CPU, serialización explícita de preprocesadores y modelos, y clientes HTTP livianos para consultar Prometheus sin introducir capas

intermedias. La tabla 2.1 resume las bibliotecas principales y el rol que cumplen dentro del sistema.

TABLA 2.1. Bibliotecas de terceros utilizadas en el sistema y su rol.

Biblioteca	Versión	Rol en el sistema
TensorFlow / Keras	2.8	Definición y entrenamiento del autoencoder LSTM.
scikit-learn	1.0	Normalización (MinMaxScaler) con cotas fijas.
joblib	1.1	Persistencia del preprocesador entrenado.
NumPy	1.19	Operaciones con arreglos n-dimensionales y cálculo del error de reconstrucción.
pandas	1.3	Manipulación de series temporales y construcción de ventanas.
requests	2.26	Consultas HTTP a la API de Prometheus.
prometheus_client	0.17	Exposición de métricas en el servicio simulado.
PyYAML	5.4	Carga de archivos de configuración.
matplotlib / seaborn	3.4 / 0.11	Gráficos de evaluación offline del modelo.

En la práctica, las dependencias se agruparon en tres bloques funcionales y modulares: cómputo numérico y aprendizaje profundo (TensorFlow, NumPy, pandas, scikit-learn), observabilidad (requests, prometheus_client, PyYAML) y evaluación visual (matplotlib, seaborn). Esta separación facilitó auditar qué componentes intervienen en el entrenamiento, en la inferencia y en los ensayos documentados en el capítulo de resultados.

2.2. Datasets y datos de entrenamiento

El conjunto de datos utilizado para entrenar y evaluar el sistema corresponde a telemetría operativa del servicio de TV-over-IP de la empresa cliente, recolectada con un intervalo de muestreo de 30 segundos sobre una ventana histórica de 90 días. La fuente primaria es la instancia interna de Prometheus de la organización [4, 5]. Las series se obtuvieron mediante la API HTTP de rango `api/v1/query_range`, con las consultas PromQL definidas en el archivo `config/data.yaml` del repositorio.

2.2.1. Confidencialidad y entorno de demostración

El entrenamiento sobre telemetría real se realizó dentro de la infraestructura autorizada de la empresa. No obstante, las métricas de desempeño operativo (carga, latencia, errores y consumo de recursos) constituyen información confidencial: su publicación en un repositorio académico o en capturas de la memoria permitiría inferir aspectos del comportamiento productivo del servicio.

Por ese motivo, el material reproducible del trabajo, incluido el repositorio entregable y las figuras ilustrativas de este capítulo, emplea un entorno demostrativo

con un servicio simulado y un generador sintético que replica la misma familia de métricas y la semántica de agregación PromQL del entorno productivo.

Esta separación no implica que el modelo se haya entrenado únicamente con datos sintéticos: el régimen nominal utilizado para el aprendizaje provino de Prometheus interno. El simulador permitió desarrollar y validar el *pipeline* de forma reproducible, ejecutar demostraciones sin acceso a la red de producción y documentar el sistema sin exponer series identificables para los fines académicos de este trabajo.

2.2.2. Datasets de entrenamiento utilizados

Al tratarse de telemetría propia de un servicio en producción, no se utilizaron *datasets* públicos como fuente principal. El dominio (calidad de servicio en *streaming* sobre infraestructura específica) no encuentra equivalencia directa en repositorios abiertos como Numenta NAB [22] o Yahoo Webscope [23], que agrupan series heterogéneas con distinto muestreo y sin el stack Prometheus del cliente.

2.3. Modelos preentrenados y transferencia de aprendizaje

El sistema no incorporó modelos preentrenados ni esquemas de *transfer learning*. La razón es triple. En primer lugar, no existen modelos públicamente disponibles entrenados sobre telemetría operativa de servicios de TV-over-IP con la misma combinación de métricas y régimen de muestreo de 30 segundos. La transferencia desde dominios distintos (texto, visión o series financieras) no resultaba directamente aplicable porque las entradas, la escala temporal y la semántica de las variables difieren sustancialmente.

En segundo lugar, *benchmarks* genéricos como Numenta NAB entrenan o evalúan sobre series univariadas o multivariadas heterogéneas que no coinciden con el vector de once variables construido en este trabajo (cinco métricas de servicio y seis atributos temporales). Un modelo preentrenado en esos conjuntos no habría capturado las correlaciones entre latencia, tasa de errores y patrones circadianos del servicio concreto.

En tercer lugar, el volumen disponible de datos del régimen nominal resultó adecuado para un entrenamiento desde cero en tiempos razonables sobre hardware modesto, sin necesidad de inicializaciones de gran capacidad. Ante la ausencia de un punto de partida compatible y la suficiencia de datos propios, se entrenó el autoencoder LSTM exclusivamente sobre telemetría del entorno autorizado.

2.4. Infraestructura de cómputo

El sistema fue concebido para ejecutarse en infraestructura propia, sin envío de telemetría a servicios externos. Tanto el entrenamiento como la inferencia se realizaron en CPU. El tamaño reducido del modelo y de la ventana de inferencia hicieron innecesario el uso de aceleradores gráficos.

El entorno de desarrollo se orquestó con Docker y Docker Compose [24], lo que permitió desplegar de forma reproducible los servicios involucrados desde un único archivo de declaración. Se distinguieron dos perfiles de uso: el perfil *dev*, con el stack completo de demostración (servicio simulado, Prometheus, Grafana y

detector), y el despliegue productivo sobre la infraestructura AWS de la empresa cliente, donde únicamente se agregó el contenedor de detección a la red interna existente.

El detector se distribuyó como una imagen basada en `python:3.10-slim`, con un usuario sin privilegios y montajes de los directorios de modelos, configuración y registros para preservar artefactos entre reinicios. La asignación de recursos definida en `docker-compose.yml` contempló un límite de una CPU virtual y 2 GB de memoria RAM para el contenedor de detección, lo que resultó suficiente para una inferencia cada 30 segundos sobre ventanas de 10 minutos. Los servicios auxiliares (servicio simulado, Prometheus y Grafana) se aislaron en una red puente dedicada y se expusieron solo los puertos necesarios para inspección manual durante el desarrollo.

El entrenamiento se ejecutó en un equipo de desarrollo con procesador de uso general y sin GPU dedicada. El ciclo completo se completó en un tiempo del orden de minutos a decenas de minutos, según la cantidad de ventanas resultantes del período de 90 días. Esta capacidad de reentrenar en un horizonte corto fue un criterio explícito de diseño, dado que el régimen de operación del servicio puede evolucionar con cambios de versión del producto o con variaciones estacionales del tráfico.

La integración con Opsgenie para la emisión de alertas y la generación de enlaces contextuales a Grafana se realizó mediante sus APIs públicas a través de HTTPS, sin agentes adicionales en el contenedor. De este modo, la infraestructura agregada por el sistema se redujo a un único servicio adicional dentro de la red interna, lo que respetó los requerimientos de seguridad y privacidad establecidos para el trabajo.

La tabla 2.2 resume el entorno técnico bajo el que se entrenó y desplegó el sistema.

TABLA 2.2. Entorno de cómputo utilizado para entrenamiento y despliegue.

Componente	Descripción
Lenguaje y environment	Python 3.10 (<code>python:3.10-slim</code>).
Orquestación	Docker Compose, perfil dev.
Uso de recursos	1 CPU, 2 GB RAM.
Aceleración por GPU	No (solo CPU).
Servicios auxiliares	Prometheus, Grafana, servicio simulado.
Red	Puente <code>monitoring</code> , sin exposición pública.
Destino de deploy	AWS propia, sin analítica de terceros.

2.5. Métricas de evaluación

La evaluación del sistema combinó dos planos complementarios. En el primero se valoró la calidad de reconstrucción del autoencoder, base del aprendizaje no supervisado, mediante el error cuadrático medio (MSE) [12] como función de pérdida y señal escalar de anomalía, y el error absoluto medio (MAE) como métrica auxiliar de monitoreo.

En el segundo se valoró la capacidad de detección sobre ventanas con anomalías sintéticas inyectadas, y para ello se emplearon métricas de clasificación binaria habituales [13]: precisión, *recall*, *F1-score* y tasa de falsos positivos.

La distinción entre ambos planos respondió al carácter no supervisado del entrenamiento: las métricas de clasificación solo pudieron computarse en escenarios controlados con anomalías conocidas, mientras que el error de reconstrucción se midió de forma continua sobre tramos nominales.

Las métricas utilizadas y el propósito que cumplieron en la validación fueron los siguientes:

- MSE: error cuadrático medio entre la ventana de entrada y su reconstrucción; función de pérdida del entrenamiento y señal escalar de anomalía en inferencia.
- MAE: error absoluto medio entre entrada y reconstrucción; métrica auxiliar de monitoreo durante el entrenamiento, menos sensible a valores atípicos.
- Precisión: fracción de ventanas marcadas como anómalas que efectivamente lo eran; controla la proporción de falsas alertas elevadas al servicio de guardia.
- Recall: fracción de anomalías reales correctamente detectadas; mide la cobertura de incidentes ante anomalías sintéticas inyectadas.
- F1-score: media armónica de precisión y exhaustividad; resume en un único valor el compromiso entre falsas alertas y cobertura.
- Tasa de falsos positivos: fracción de ventanas nominales marcadas erróneamente como anómalas; verifica la estabilidad del umbral sobre tramos de régimen nominal.

El procedimiento concreto de cálculo de estas métricas, la calibración del umbral sobre la distribución de errores de validación y los resultados cuantitativos obtenidos se detallan en los capítulos de diseño y de resultados.

Capítulo 3

Diseño e implementación

En este capítulo se describe el diseño e implementación del sistema de detección de anomalías: el pipeline de datos desde Prometheus hasta las ventanas multivariadas, la arquitectura del autoencoder LSTM, el procedimiento de entrenamiento y calibración del umbral, y la integración del servicio de inferencia con Opsgenie y Grafana.

3.1. Pipeline de datos

El pipeline de datos automatiza las etapas que transforman telemetría cruda en secuencias listas para el modelo.

Se implementó como un conjunto de módulos Python invocados desde el script de entrenamiento (`scripts/train.py`) y reutilizados en inferencia (`scripts/inference.py`), de modo que entrenamiento y producción comparten exactamente las mismas reglas de transformación.

La configuración operativa, incluidas las consultas PromQL, las cotas de normalización, el tamaño de ventana y los parámetros de alerta, se externalizó en archivos YAML bajo `config/`, lo que permitió ajustar el comportamiento sin modificar el código fuente.

3.1.1. Recolección y alineación temporal

La primera etapa obtiene series desde la API HTTP de rango de Prometheus (`api/v1/query\_range`).

Un cliente dedicado (`PrometheusClient`) lee las consultas definidas en `config/data.yaml` y ejecuta cada consulta con paso de muestreo de 30 segundos.

Tras obtener las series por instancia, se agregaron los resultados según la semántica de cada métrica:

- `sum`: contadores y tasas (`request_rate`, `error_rate`, `cpu_usage`).
- `max`: percentiles de latencia (`latency_p95`).
- `mean`: gauges de memoria (`memory_usage`).

Las series devueltas se integraron en un marco temporal común, con `timestamp` como índice.

Cuando faltaban puntos aislados dentro del intervalo de muestreo, se aplicó *forward-fill* acotado antes de descartar ventanas incompletas. El cliente incorporó además un ajuste automático del paso de consulta cuando el rango solicitado superaba el límite de 11 000 puntos por petición impuesto por Prometheus y preservó la cobertura temporal sin truncar silenciosamente el historial.

Para el entrenamiento sobre el entorno autorizado de la empresa, el cliente apuntó a la instancia Prometheus interna. En el repositorio reproducible y en el perfil dev de Docker Compose, las mismas consultas se ejecutaron contra un servicio simulado instrumentado con `prometheus\_client`, compartiendo núcleo de generación con el generador sintético offline (`traffic\_simulation\_core.py`).

Así se garantizó paridad semántica entre fuentes sin exponer telemetría confidencial en el material académico reproducible.

La tabla 3.1 detalla las consultas PromQL configuradas, la agregación aplicada tras obtener las series por instancia y la interpretación operativa de cada señal. Mantener las consultas declarativas permitió auditar qué aspecto del servicio alimenta cada variable del modelo y replicar el mismo criterio en el simulador.

TABLA 3.1. Consultas PromQL y agregación por métrica del servicio.

Variable	Consulta PromQL (resumida)	Agreg.	Interpretación
<code>request_rate</code>	<code>rate(http_requests_total[5m])</code>	sum	Demanda instantánea sobre el servicio
<code>latency_p95</code>	<code>histogram_quantile(0.95, rate(...[5m]))</code>	max	Cola superior de tiempos de respuesta
<code>memory_usage</code>	<code>process_resident_memory_bytes{...}</code>	mean	Presión de memoria del proceso
<code>error_rate</code>	<code>rate(http_request_errors_total[5m])</code>	sum	Tasa de respuestas erróneas
<code>cpu_usage</code>	<code>rate(process_cpu_seconds_total[5m])</code>	sum	Consumo de CPU del servicio

El intervalo de cinco minutos en las funciones `rate` y `histogram_quantile` suavizó fluctuaciones de muestreo periódico sin ocultar degradaciones sostenidas. Ese valor se mantuvo coherente con las reglas de alerta existentes en la organización, lo que facilitó contrastar el detector aprendido con umbrales declarativos en evaluaciones posteriores.

Cuando el historial solicitado superaba el límite de puntos por consulta, el cliente recalculó el paso efectivo manteniendo fijo el rango temporal total. De este modo, un entrenamiento sobre 90 días no truncó silenciosamente las primeras semanas del período, situación que se detectó en pruebas iniciales y que habría sesgado la estimación de estacionalidad semanal. La resolución efectiva resultante fue más gruesa solo cuando fue estrictamente necesaria para cumplir la restricción de la API.

Se eligió un horizonte de 90 días porque cubre varios ciclos semanales completos y proporciona del orden de 259 200 muestras por métrica, volumen suficiente para estimar un régimen nominal estable sin exigir almacenamiento excesivo. El particionado reservó el 20 % final de las ventanas para validación y preservó el orden cronológico a fin de evitar filtraciones de información futura.

3.1.2. Ingeniería de variables y normalización

Tras la alineación, el módulo `DataPreprocessor` enriqueció cada instante con seis variables temporales derivadas del reloj de la muestra:

1. `hour_sin` y `hour_cos`: codificación cíclica de la hora del día mediante $\sin(2\pi h/24)$ y $\cos(2\pi h/24)$.
2. `dayofweek_sin` y `dayofweek_cos`: codificación cíclica del día de la semana (0–6).
3. `is_weekend`: indicador binario para sábado y domingo.
4. `is_night`: indicador binario para el intervalo 22:00–06:00.

La codificación cíclica evita discontinuidades artificiales en los límites del calendario (por ejemplo, entre las 23:00 y las 00:00) y permite al modelo distinguir patrones diarios y semanales sin depender de umbrales fijos por horario.

La normalización se realizó con `MinMaxScaler` en modo `fixed_minmax`: en lugar de ajustar mínimos y máximos sobre el conjunto de entrenamiento, se fijaron cotas determinísticas por variable en `config/data.yaml`, derivadas del análisis del servicio simulado y validadas contra consultas reales. La tabla 3.2 resume los rangos adoptados.

En las primeras iteraciones se probó `StandardScaler`, que memoriza media y desviación estándar del lote de entrenamiento. Cuando el modelo entrenado con datos sintéticos se evaluó sobre Prometheus productivo, las series normalizadas quedaron desplazadas respecto del dominio aprendido y el error de reconstrucción superó el umbral de forma sistemática en tramos nominales.

El modo `fixed_minmax` eliminó ese acoplamiento: la misma observación cruda produce siempre la misma coordenada normalizada, con independencia del origen o del período de entrenamiento. Este criterio resultó central para desplegar un único par modelo–preprocesador en producción sin recalibrar el escalador ante cada reentrenamiento.

Las cotas de la tabla 3.2 se derivaron del análisis del servicio simulado que se basó en el servicio real como referencia directa:

TABLA 3.2. Cotas de normalización `fixed_minmax` por variable.

Variable	Rango mínimo	Rango máximo
<code>request_rate</code>	0 req/s	150 req/s
<code>latency_p95</code>	0 s	0,50 s
<code>memory_usage</code>	0 B	2×10^9 B
<code>error_rate</code>	0 err/s	3,0 err/s
<code>cpu_usage</code>	0	0,15
Variables temporales cíclicas	−1	1
Indicadores binarios	0	1

La tasa de peticiones sigue un factor diario sinusoidal con ruido gaussiano, la latencia percentil 95 se modela como proporcional a ese factor más un piso constante, y las tasas de error y CPU escalan con la demanda esperada. Los límites superiores incorporaron margen por encima del máximo observado en condiciones nominales para absorber picos breves sin saturar la escala. Si una observación

excediera una cota, `MinMaxScaler` la acotó al intervalo $[0, 1]$, señal explícita de régimen extremo que el autoencoder aprendió a reconstruir con mayor error.

El preprocesador serializado (`models/preprocessor.joblib`) persiste el escalador, la lista de columnas activas y la configuración de variables temporales, de modo que inferencia y entrenamiento aplican la misma transformación bit a bit.

3.1.3. Construcción de ventanas deslizantes

La etapa final del pipeline convirtió la serie multivariada preprocesada en tensores tridimensionales (N, T, F) , donde N es la cantidad de ventanas, $T = 20$ el número de pasos temporales y $F = 11$ el número de variables por paso. La clase `WindowGenerator` implementó dos modos:

- Entrenamiento (`create_sequences`): ventanas deslizantes con $stride = 1$, con solapamiento máximo entre muestras consecutivas. Con 90 días de muestreo cada 30 segundos, se produjeron alrededor de 250 000 ventanas, frente a unas 12 000 obtenidas con $stride = 20$ en prototipos iniciales.
- Inferencia (`create_single_window`): extracción de los últimos T puntos disponibles. Si la cantidad era insuficiente, el servicio omitía el ciclo de detección en lugar de rellenar con ceros, para evitar falsos positivos por ventanas artificialmente completadas.

Cada ventana abarcó 10 minutos de operación (20×30 s), horizonte elegido por equilibrar sensibilidad a degradaciones sostenidas y costo de cómputo en CPU. El particionado entre entrenamiento y validación reservó el 20 % final de las ventanas y preservó el orden cronológico.

La longitud de 20 pasos implicó que una anomalía puntual de un solo muestreo quedara diluida en la reconstrucción global de la ventana, mientras que desvíos sostenidos durante varios minutos, el caso operativo de interés, modificaron de forma coherente múltiples pasos y varias variables a la vez y elevaron el error agregado por encima del umbral. Ventanas más cortas (10 pasos) reaccionaron antes pero fueron más sensibles al ruido. Ventanas de 30 pasos (15 minutos) suavizaron en exceso incidentes breves. El valor 20 resultó del benchmark interno documentado en `tune\_model.py`.

La tabla 3.3 detalla entradas, salidas y artefactos de cada etapa, la figura 3.1 demuestra gráficamente la ventana deslizante.

TABLA 3.3. Etapas del pipeline de datos y artefactos producidos.

Etapas	Entrada / salida	Implementación
Recolección	Consultas PromQL $\rightarrow$ series por métrica	<code>PrometheusClient</code> , <code>config/data.yaml</code>
Alineación	Series $\rightarrow$ marco temporal unificado	<code>pandas</code> , <code>forward-fill</code> acotado
Ingeniería	Marco + reloj $\rightarrow$ 11 variables	<code>add_temporal_features</code> de <code>preprocessor</code>
Normalización	Crudos $\rightarrow [0, 1]$ fijo	<code>MinMaxScaler</code> , cotas YAML
Ventanas	Serie $\rightarrow (N, 20, 11)$	<code>WindowGenerator</code>
Persistencia	Escalador y metadatos	<code>preprocessor.joblib</code>

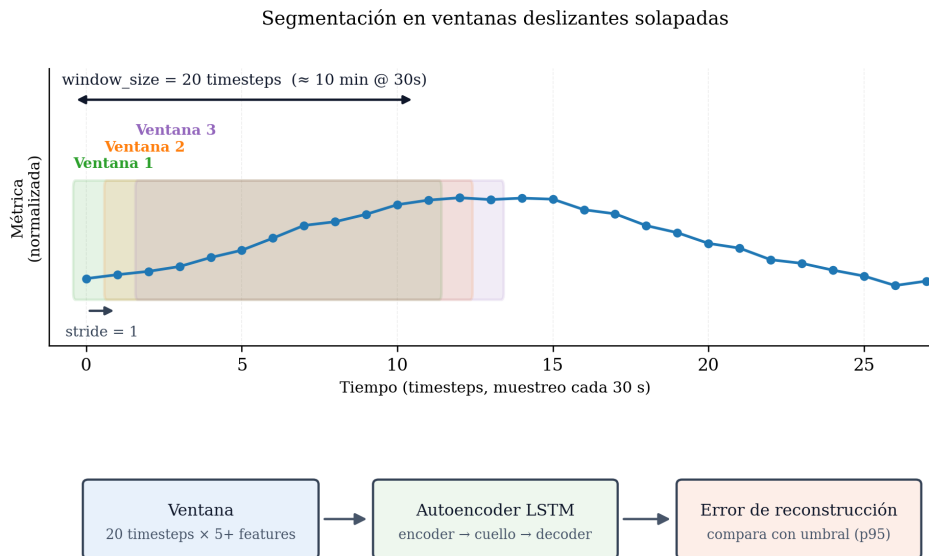


FIGURA 3.1. Estructura lógica de ventanas deslizantes.

3.1.4. Organización del código fuente

El repositorio entregable agrupó la lógica en cuatro paquetes bajo `src/`, más scripts ejecutables en `scripts/` y configuración en `config/`. La tabla 3.4 resume la responsabilidad de cada módulo principal. Esta separación permitió probar el preprocesamiento y la generación de ventanas de forma aislada antes de integrarlos con el servicio de inferencia.

TABLA 3.4. Módulos implementados y responsabilidad funcional.

Módulo / script	Responsabilidad
PrometheusClient	Consultas HTTP, agregación y alineación de series
DataPreprocessor	Variables temporales y normalización
Windowing	Ventanas deslizantes para entrenamiento e inferencia
SyntheticData	Generación offline de series de respaldo
LSTMAutoencoder	Definición, entrenamiento y persistencia del modelo
Detector	Puntuación de anomalía por ventana
OpsgenieClient	Emisión de alertas y notificaciones de resolución
GrafanaLinks	Construcción de URLs contextuales al panel de Grafana
train.py	Orquestación del entrenamiento extremo a extremo
inference.py	Bucle de detección en tiempo de operación
tune_model.py	Exploración de arquitecturas sin afectar artefactos productivos

3.1.5. Paridad entre entrenamiento e inferencia

Un requisito no funcional central del diseño fue garantizar que una ventana procesada en producción fuera indistinguible, desde el punto de vista del modelo, de una ventana equivalente obtenida durante el entrenamiento.

Cualquier divergencia en el orden de transformaciones, en las columnas activas o en la política de ventanas se tradujo en errores de reconstrucción artificialmente elevados y en falsos positivos masivos, fenómeno reproducido al comparar un escalado adaptativo entrenado con datos sintéticos contra consultas reales a Prometheus. Para cerrar esa brecha se adoptaron tres medidas concretas.

Primero, el mismo objeto `DataPreprocessor` serializado se cargó en inferencia sin reajuste sobre datos recientes. Segundo, las consultas PromQL y las reglas de agregación se leyeron del mismo archivo `config/data.yaml` en ambos modos. Tercero, la generación de ventanas en inferencia utilizó exclusivamente los últimos T puntos completos, mientras que en entrenamiento se generó el conjunto completo con `stride = 1`, pero aplicando la misma función de extracción por ventana.

3.2. Arquitectura del sistema

El sistema se organizó en capas acopladas al monitoreo existente. Un servicio simulado (en el entorno demostrativo) o la instancia productiva exponían métricas que Prometheus recolectaba periódicamente y consultaba el contenedor de detección. Grafana visualizó las series y Opsgenie recibió las alertas emitidas. La figura 3.2 resume esta topología en el perfil `dev` de Docker Compose.

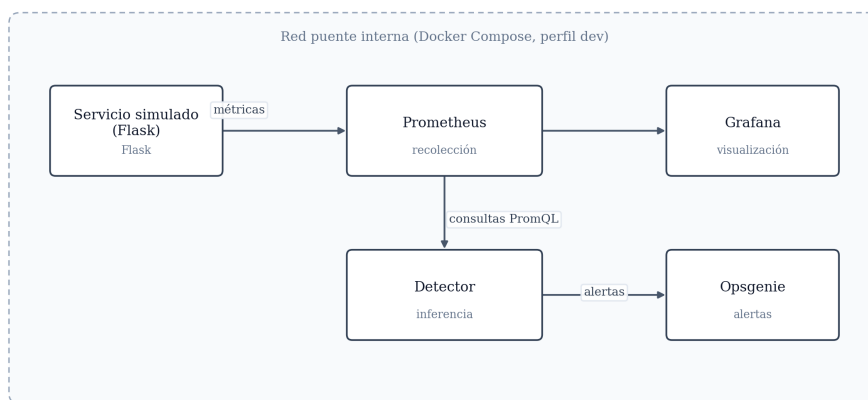


FIGURA 3.2. Topología de despliegue del sistema de detección.

El bloque detector de la figura anterior agrupa el pipeline de datos, el autoencoder LSTM y la lógica de umbral descritos en las secciones anteriores.

La figura 3.3 detalla el flujo interno de inferencia: preprocesamiento de la ventana, codificación y decodificación, cálculo del error de reconstrucción y comparación con el umbral calibrado; ante una anomalía confirmada, se emiten alertas hacia Opsgenie y enlaces contextuales a Grafana.

Las etapas `Encoder LSTM` y `Decoder LSTM` implementan el esquema codificador-*bottleneck*-decodificador de Malhotra et al. en el módulo `LSTMAutoencoder`, adaptado al vector de once variables y a la ventana de 20 pasos.

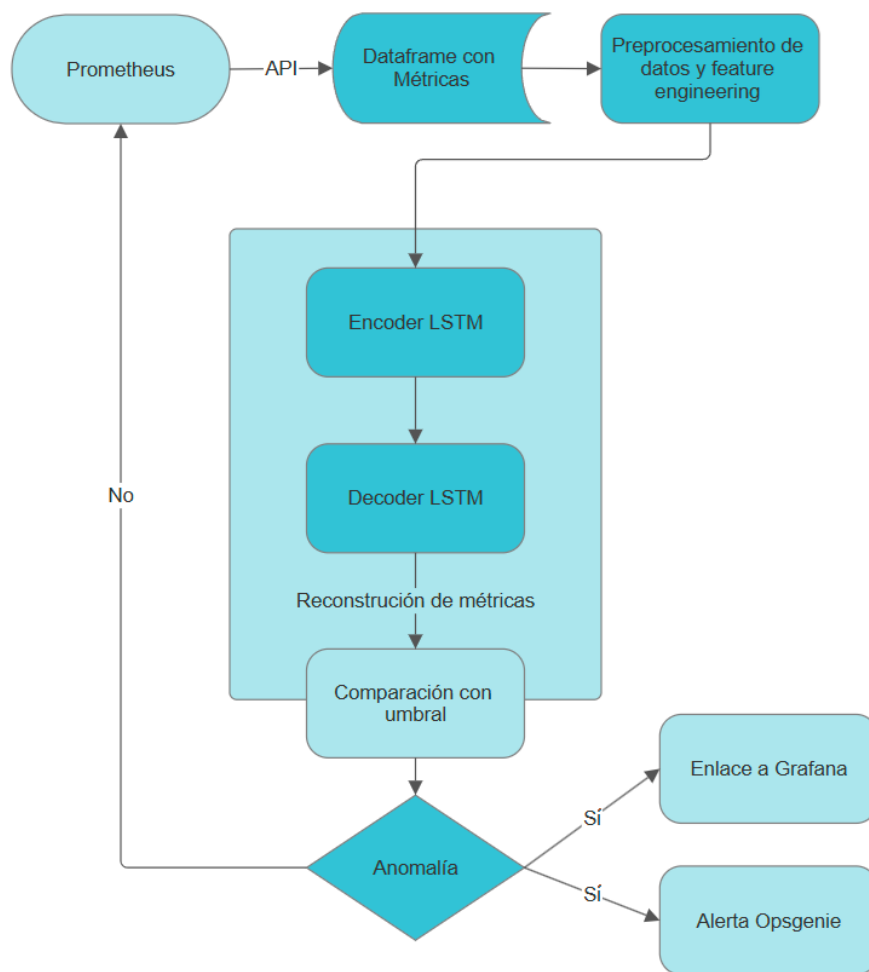


FIGURA 3.3. Flujo interno del contenedor de detección: desde la consulta a Prometheus hasta la emisión de alertas.

3.2.1. Encoder-Decoder LSTM

El codificador apiló tres capas `LSTM` con 64, 32 y 16 unidades respectivamente. Las dos primeras devolvieron secuencias completas (`return_sequences=True`), y la tercera produjo el vector latente de dimensión 16. Entre capas recurrentes se insertó `dropout` del 10 % para regularizar.

El decodificador expandió el latente mediante `RepeatVector`, replicándolo a lo largo de los 20 pasos temporales, y aplicó tres capas `LSTM` simétricas (16, 32 y 64 unidades) con `dropout` del 10 %. La salida se obtuvo con una capa `TimeDistributed(Dense(11))` que reconstruyó las once variables en cada instante. La figura 3.4 resume la pila de capas y las formas tensoriales intermedias.

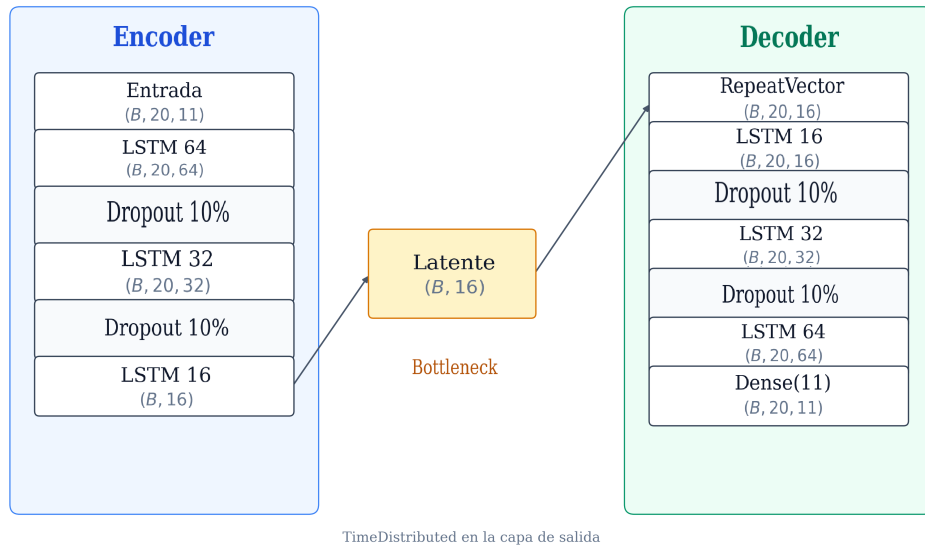


FIGURA 3.4. Arquitectura del autoencoder LSTM: codificador, *bottleneck* latente y decodificador.

La tabla 3.5 complementa el diagrama con el rol de cada capa.

TABLA 3.5. Capas del autoencoder LSTM y formas intermedias (B = tamaño de lote).

Capa	Forma de salida	Rol
Entrada	$(B, 20, 11)$	Ventana multivariada normalizada
LSTM codificador 64	$(B, 20, 64)$	Captura dependencias locales
LSTM codificador 32	$(B, 20, 32)$	Compresión progresiva
LSTM codificador 16	$(B, 16)$	Representación latente
RepeatVector	$(B, 20, 16)$	Expansión temporal del latente
LSTM decodificador 16-64	$(B, 20, 64)$	Reconstrucción secuencial
Dense por paso	$(B, 20, 11)$	Salida reconstruida

3.2.2. Función de pérdida y señal de anomalía

El entrenamiento minimizó el error cuadrático medio (MSE) [12] entre la ventana de entrada y su reconstrucción, promediado sobre los 20 pasos y las 11 variables. Como métrica auxiliar de monitoreo se registró el error absoluto medio (MAE) [12]. En inferencia, la señal escalar de anomalía fue el MSE por ventana, definido en la ecuación (3.1):

$$e = \frac{1}{T \cdot F} \sum_{t=1}^T \sum_{f=1}^F (x_{t,f} - \hat{x}_{t,f})^2 \quad (3.1)$$

donde x es la ventana observada y $\hat{x}$ la reconstrucción del modelo. Una ventana se marcó como anómala cuando e superó un umbral τ calibrado sobre errores de validación.

Además del criterio binario, se definió una magnitud de confianza relativa para priorizar alertas, según la ecuación (3.2):

$$c = \frac{e - \tau}{\tau} \quad \text{cuando } e > \tau \quad (3.2)$$

y $c = 0$ en caso contrario. Valores de c más altos indicaron un apartamiento más marcado del régimen nominal aprendido.

Esos valores se utilizaron para asignar prioridad en Opsgenie (tabla 3.6).

TABLA 3.6. Asignación de prioridad Opsgenie según confianza de la anomalía.

Prioridad	Umbral de c	Interpretación operativa
P1	$c > 2,0$	Desvío muy marcado; respuesta inmediata.
P2	$c > 1,0$	Degradación significativa; escalamiento si persiste.
P3	$c > 0,5$	Anomalía moderada; revisión en guardia.
P4	$c \leq 0,5$	Registro sin escalamiento urgente.

3.2.3. Acoplamiento multivariado y espacio latente

A diferencia de reglas univariadas por métrica, el autoencoder procesa las once variables de forma conjunta en cada paso temporal. El codificador LSTM mantiene un estado oculto que acumula contexto a lo largo de los 20 pasos, de modo que una subida aislada de latencia en presencia de demanda estable y error nulo puede reconstruirse con bajo error, mientras que una combinación de incremento simultáneo en `error_rate`, `latency_p95` y `cpu_usage` eleva e aunque ninguna variable supere por sí sola un umbral fijo predefinido.

El vector latente de dimensión 16 actúa como *bottleneck*: obliga al modelo a capturar regularidades compactas del régimen nominal (ciclos diarios, diferencia fin de semana vs. día hábil, correlación entre carga y latencia) y descartar variaciones de alta frecuencia no repetibles. Durante el entrenamiento, la pérdida de validación descendió de forma monótona hasta estabilizarse en valores del orden de 10^{-3} en la escala normalizada, señal de que el bottleneck conservó información suficiente para reconstruir ventanas nominales sin memorizar ruido puntual. La tabla 3.7 resume la complejidad paramétrica y el costo operativo.

TABLA 3.7. Complejidad del modelo y costo operativo estimado.

Atributo	Valor o estimación
Parámetros entrenables	Del orden de 10^5 (capas LSTM + densas)
Entrada por inferencia	1 ventana (20, 11)
Tiempo por ciclo de detección	< 1 s en contenedor con 1 vCPU
Memoria del contenedor de detección	Límite 2 GB. Uso típico inferior a 512 MB
Frecuencia del bucle	1 evaluación cada 30 s

3.2.4. Decisiones de diseño arquitectónico

Se eligió una topología simétrica 64–32–16–32–64 por ser suficientemente expresiva para once variables en CPU y mantener tiempos de inferencia acotados (inferior a un segundo por ciclo en el contenedor de despliegue). Configuraciones con bottleneck más estrecho (8 unidades) y más ancho (32 unidades) se evaluaron en el script `tune\_model.py`. La topología base ofreció el mejor equilibrio entre pérdida de validación y estabilidad frente a datos de Prometheus.

La activación lineal en la capa de salida preservó la interpretabilidad de las variables reconstruidas en la escala normalizada. No se emplearon capas convolucionales ni mecanismos de atención: el muestreo fijo cada 30 segundos y la ventana corta de 10 minutos hicieron innecesaria una mayor complejidad para capturar la estacionalidad diaria modelada explícitamente mediante variables temporales.

Los pesos entrenados y la configuración estructural se serializaron por separado (`lstm\_autoencoder.weights.h5` y `lstm\_autoencoder\_config.json`) para reconstruir el grafo computacional en inferencia sin acoplarse a una versión concreta del entorno de entrenamiento.

3.3. Entrenamiento, evaluación y optimización

Esta sección documenta el procedimiento de entrenamiento del autoencoder, la calibración del umbral de detección y la exploración acotada de hiperparámetros alternativos.

La evaluación del sistema se concibió en función de indicadores operativos verificables: tasa de falsos positivos sobre tramos considerados nominales, capacidad de señalar degradaciones acordadas con el equipo de operaciones y tiempo entre el inicio de un patrón anómalo y la emisión de la alerta. Esos criterios orientan los ensayos del capítulo correspondiente y permiten contrastar el enfoque aprendido frente a líneas base por umbrales fijos.

3.3.1. Características del dataset

Cada observación temporal del conjunto consta de cinco métricas técnicas del servicio, seleccionadas para cubrir carga, experiencia del usuario y salud del proceso:

1. `request_rate`: tasa de peticiones HTTP por segundo.
2. `latency_p95`: percentil 95 de la latencia de respuesta.
3. `memory_usage`: memoria residente del proceso.
4. `error_rate`: tasa de respuestas erróneas por segundo.
5. `cpu_usage`: fracción de CPU consumida.

Sobre estas métricas se derivaron seis variables temporales adicionales mediante codificación cíclica de la hora y del día de la semana, junto con indicadores de fin de semana y horario nocturno. La tabla 3.8 sintetiza las características del conjunto antes y después de la transformación a ventanas deslizantes de 20 pasos (10 minutos).

TABLA 3.8. Descripción del conjunto de datos utilizado para entrenamiento y evaluación.

Atributo	Valor o descripción
Origen de datos	Prometheus (instancia interna) con simulador y generador sintético de núcleo compartido para desarrollo.
Confidencialidad	Telemetría productiva no publicada; figuras ilustrativas con datos sintéticos.
Período cubierto	90 días corridos previos al entrenamiento.
Intervalo de muestreo	30 segundos.
Cantidad aproximada de muestras	259 200 puntos por métrica.
Métricas técnicas	request_rate, latency_p95, memory_usage, error_rate y cpu_usage.
Variables temporales derivadas	hour_sin, hour_cos, dayofweek_sin, dayofweek_cos, is_weekend e is_night.
Dimensión de cada ventana	20 pasos $\times$ 11 variables.
Longitud temporal de la ventana	10 minutos.
Particionado entrenamiento / validación	80 % / 20 % en orden temporal (último tramo como validación).
Etiquetas	Régimen no supervisado: entrenamiento sobre tramos nominales.

La figura 3.5 muestra la distribución marginal de las cinco métricas técnicas sobre datos sintéticos ilustrativos con estacionalidad diaria compatible con el *mock service*. Se incluye únicamente con fines demostrativos, dado que no corresponde publicar histogramas de la telemetría real.

Distribución de métricas técnicas (datos sintéticos ilustrativos)

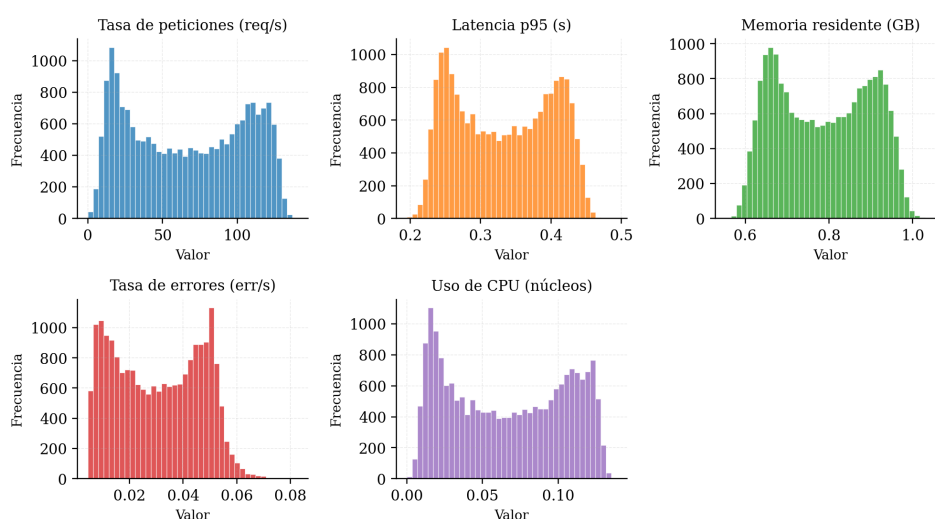


FIGURA 3.5. Distribución de las métricas técnicas del servicio (datos sintéticos ilustrativos).

Cabe aclarar que el problema se planteó como detección no supervisada: el conjunto de entrenamiento se compuso de tramos en los que el servicio operó dentro

de su régimen nominal y no se dispuso de un etiquetado fiable de anomalías históricas a la escala del dataset completo.

3.3.2. Procedimiento de entrenamiento

El script `scripts/train.py` orquestó el flujo completo: carga de configuración, obtención de datos, preprocesamiento, generación de ventanas, entrenamiento, persistencia de artefactos y cálculo del umbral. Los hiperparámetros principales se fijaron en `config/model.yaml` y se resumen en la tabla 3.9.

TABLA 3.9. Hiperparámetros de entrenamiento del autoencoder LSTM.

Parámetro	Valor
Optimizador	Adam, tasa de aprendizaje 0,001
Función de pérdida	MSE
Tamaño de lote (<i>batch</i>)	256
Épocas máximas	30
Parada temprana	Paciencia 10 épocas sobre <code>val_loss</code>
Reducción de tasa de aprendizaje	Factor 0,5 cuando <code>val_loss</code> se estanca
Partición entrenamiento / validación	80 % / 20 % temporal
Dropout entre capas LSTM	0,1

El optimizador Adam actualizó los pesos minimizando la MSE entre entrada y salida del autoencoder (objetivo clásico de reconstrucción). Se registraron `EarlyStopping` con restauración de los mejores pesos y `ReduceLROnPlateau` para reducir la tasa de aprendizaje cuando la pérdida de validación dejó de mejorar.

Con ello se aceleró la convergencia en las últimas épocas sin sobreajustar ruido de corto plazo.

El entrenamiento se ejecutó íntegramente en CPU sobre un historial de 90 días con muestreo cada 30 segundos. El ciclo completo, incluida la generación de ventanas solapadas, se completó en un tiempo del orden de decenas de minutos en el equipo de desarrollo, lo que habilitó reentrenamientos periódicos ante cambios de régimen operativo.

Durante el ajuste, el modelo no recibió etiquetas de anomalía: cada ventana de entrenamiento se utilizó simultáneamente como entrada y como objetivo de reconstrucción. Esta formulación no supervisada es coherente con la ausencia de un catálogo fiable de incidentes históricos a escala del dataset completo. El conjunto de validación temporal cumplió una doble función: guiar la parada temprana durante el aprendizaje y proporcionar la distribución de errores sobre la que se calibró el umbral de detección.

El procedimiento de entrenamiento puede resumirse en ocho pasos ejecutados de forma secuencial por `train.py`:

1. Lectura de configuración YAML.
2. Obtención del historial desde Prometheus o, en su defecto, generación sintética de respaldo.
3. Validación de cantidad mínima de filas.

4. Preprocesamiento y persistencia del escalador.
5. Construcción de ventanas con `stride = 1`.
6. Partición temporal 80/20.
7. Ajuste del autoencoder con *callbacks* de parada temprana y reducción de tasa de aprendizaje.
8. Cálculo del umbral sobre errores de validación y guardado de artefactos en `models/`.

Cualquier interrupción antes del paso 8 deja el directorio en estado inconsistente. Por ello el despliegue productivo exige verificar la presencia de los cuatro artefactos listados en la tabla 3.10.

TABLA 3.10. Artefactos del modelo y del pipeline utilizados en inferencia.

Artefacto	Contenido
<code>lstm_autoencoder.weights.h5</code>	Pesos entrenados del autoencoder
<code>lstm_autoencoder_config.json</code>	Arquitectura y metadatos de reconstrucción del grafo
<code>preprocessor.joblib</code>	Escalador <code>fixed_minmax</code> y columnas activas
<code>anomaly_threshold.npy</code>	Umbral τ calibrado en validación

3.3.3. Calibración del umbral

Tras el entrenamiento, se calcularon los errores e de la ecuación (3.1) sobre el conjunto de validación temporal y se definió el umbral τ como un percentil de esa distribución, configurable en `config/alerting.yaml`. El percentil inicial se fijó en un valor alto (99,5) para limitar falsos positivos sobre tramos nominales. El valor definitivo se ajustó iterativamente con el equipo de operaciones durante las pruebas de validación y priorizó una tasa de alertas compatible con la capacidad de respuesta del servicio de guardia.

Como alternativa implementada pero no adoptada en producción, el mismo módulo soportó un umbral por media más dos desviaciones estándar de los errores de validación. La variante por percentil resultó más interpretable para los operadores porque expresa directamente qué fracción del comportamiento nominal se tolera antes de escalar.

El umbral serializado (`models/anomaly\_threshold.npy`) se cargó en inferencia junto con el modelo y el preprocesador. Así se mantuvo coherencia entre el criterio de entrenamiento y el despliegue.

3.3.4. Exploración de hiperparámetros

Se realizó una búsqueda acotada de configuraciones alternativas mediante `scripts/tune\_model.py`, que entrenó candidatos en un directorio aislado (`evaluation/tuning\_runs/`) sin sobrescribir los artefactos productivos.

Los escenarios evaluados incluyeron bottleneck reducido ([32, 16, 8]), bottleneck ampliado ([128, 64, 32]), dropout 0,2 y ventanas de 30 pasos (15 minutos).

Los criterios de selección combinaron la pérdida de validación, el margen entre error medio en datos nominales y el umbral propuesto, y la tasa de falsos positivos sobre ventanas sintéticas con anomalías inyectadas. La tabla 3.11 resume los candidatos evaluados; a continuación se comentan los resultados de cada variante.

TABLA 3.11. Exploración de configuraciones alternativas (`tune_model.py`).

Candidato	Variante
baseline	64–32–16, ventana 20
small_bottleneck	32–16–8
wide_bottleneck	128–64–32
window_30	30 pasos (15 min)
dropout_02	Dropout 0,2

La configuración `baseline` (ventana 20, capas 64–32–16, dropout 0,1, stride 1, `fixed_minmax`) ofreció el mejor equilibrio entre validación e inferencia en Prometheus y se mantuvo como referencia productiva por combinar simplicidad de despliegue y desempeño estable. La variante `small_bottleneck` redujo la capacidad del codificador y tendió a subestimar variaciones legítimas del régimen nominal. `wide_bottleneck` incrementó el costo de CPU con una ganancia marginal en la pérdida de validación. Con `window_30`, la ventana más larga introdujo mayor inercia y retrasó la detección de incidentes breves. Por último, `dropout_02` produjo subajuste y una pérdida de validación más elevada.

Dos decisiones de preprocesamiento resultaron determinantes y se documentaron explícitamente por su impacto en la tasa de falsos positivos:

1. Sustituir el escalado adaptativo por `fixed_minmax`, lo que eliminó el desajuste entre la distribución de entrenamiento y series productivas con distinta escala absoluta.
2. Reducir el stride de ventanas de 20 a 1, lo que aumentó en un factor del orden de 20 la cantidad de ejemplos de entrenamiento y mejoró la representación de patrones circadianos en el espacio latente.

3.4. Integración e implementación del sistema

Esta sección describe el servicio de inferencia en contenedor, la gestión de alertas hacia Opsgenie, los enlaces contextuales a Grafana y el despliegue en los entornos de simulación y productivo.

3.4.1. Servicio de inferencia

El servicio de detección (`scripts/inference.py`) se ejecutó como proceso en segundo plano dentro de un contenedor Docker basado en `python:3.10-slim`. En cada ciclo, configurado cada 30 segundos en `config/data.yaml`, el servicio:

1. Obtuvo las métricas de los últimos 10 minutos desde Prometheus (o desde el generador sintético en el entorno demo).

2. Verificó que la ventana estuviera completa y sin huecos superiores al doble del intervalo de muestreo.
3. Aplicó el preprocesador persistido y construyó la ventana más reciente.
4. Calculó el error de reconstrucción con el autoencoder cargado.
5. Comparó el error con el umbral y, si correspondía, disparó el flujo de alertas.

La clase `AnomalyDetector` encapsuló los pasos de preprocesamiento, ventana y puntuación. También devolvió los valores originales y reconstruidos para diagnóstico operativo. Ante excepciones de datos, el servicio registró el fallo y continuó en el ciclo siguiente, de modo que un error puntual de red o de consulta no detuvo el monitoreo.

Antes de puntuar una ventana, el servicio verificó tres condiciones de calidad de datos:

- Cantidad mínima de puntos igual al tamaño de ventana.
- Ausencia de huecos mayores al doble del intervalo de muestreo.
- Frescura del último punto respecto del reloj del contenedor.

Si alguna fallaba, el ciclo se omitió deliberadamente. Esta salvaguarda evitó falsos positivos por relleno con ceros cuando Prometheus aún no había acumulado historial suficiente tras un reinicio o una suspensión del entorno de desarrollo.

Cada resultado de detección quedó registrado en memoria con error de reconstrucción, umbral, indicador binario de anomalía y, cuando correspondió, las series original y reconstruida por variable. Esos registros permitieron contrastar cada alerta con el comportamiento efectivo de las métricas en el intervalo afectado y respaldaron la validación cuantitativa del sistema.

3.4.2. Gestión de alertas y deduplicación

La integración con Opsgenie se implementó en `OpsgenieClient`, que envió alertas HTTPS con prioridad derivada de la confianza (exceso relativo del error sobre el umbral). Cada alerta incluyó:

- Error de reconstrucción y marca temporal.
- Comparación entre métricas observadas y reconstruidas.
- Enlace al panel de Grafana acotado al intervalo de la ventana afectada (generado por `GrafanaLinkGenerator`).

Para mitigar la fatiga de alertas se configuraron mecanismos en `config/alerting.yaml`:

- Filtrado por confianza mínima: descartar ciclos cuyo exceso sobre el umbral fuera inferior al 25 %.
- Confirmación por ciclos consecutivos: exigir dos ciclos anómalos consecutivos antes de abrir una alerta nueva.
- Limitación de frecuencia: intervalo mínimo de cinco minutos entre alertas Opsgenie del mismo incidente.

- Deduplicación de incidentes: seguimiento con identificador único mientras el error permanece en un rango de tolerancia ($\pm 20\%$) respecto del valor inicial.
- Escalamiento y resolución: realerta periódica si la anomalía persiste más de 30 minutos y notificación de cierre cuando el error vuelve bajo el umbral.

Estos controles se definieron en colaboración con operaciones para equilibrar sensibilidad y volumen de notificaciones.

3.4.3. Enlaces contextuales a Grafana

El generador `GrafanaLinkGenerator` construyó URLs al `dashboard tv-metrics-dashboard` centradas en la marca temporal de detección. Cada enlace incluyó parámetros `from` y `to` en milisegundos Unix, abarcando ± 15 minutos alrededor del evento, y una anotación codificada (`var-annotation`) para facilitar la localización visual del instante señalado.

El operador que recibió la alerta en Opsgenie accedió con un clic a la misma ventana temporal que el modelo utilizó para calcular el error, sin reconstruir manualmente el intervalo en Grafana.

3.4.4. Despliegue y configuración

En el perfil `dev` de Docker Compose, el detector comparte red puente con un servicio simulado, Prometheus y Grafana. En producción, el contenedor de detección se integró a la red interna existente, consultó directamente la instancia Prometheus corporativa y emitió alertas hacia Opsgenie, sin exponer puertos hacia internet ni depender de servicios analíticos externos.

El contenedor se construyó con un `Dockerfile` multi-etapa mínimo: instalación de dependencias Python desde `requirements.txt`, copia del código fuente y arranque de `inference.py` con usuario sin privilegios. Los directorios `models/`, `config/` y `logs/` se montaron como volúmenes para permitir actualizar pesos y umbrales sin reconstruir la imagen. Los límites de recursos (1 CPU virtual y 2 GB de RAM) resultaron suficientes para el ciclo de 30 segundos en las pruebas de despliegue realizadas.

La tabla 3.12 resume los archivos de configuración que parametrizan el sistema sin recompilar la imagen.

TABLA 3.12. Archivos de configuración YAML del sistema.

Archivo	Parámetros principales
<code>config/data.yaml</code>	URL de Prometheus, consultas PromQL, cotas de normalización, intervalos de muestreo
<code>config/windowing.yaml</code>	Tamaño de ventana (20) y stride de entrenamiento (1)
<code>config/model.yaml</code>	Capas del autoencoder, hiperparámetros de entrenamiento
<code>config/alerting.yaml</code>	Método de umbral, credenciales Opsgenie, reglas de deduplicación y enlaces Grafana

La tabla 3.13 contrasta el perfil `dev` de Docker Compose con el despliegue productivo.

TABLA 3.13. Comparación entre entorno demo y despliegue productivo.

Componente	Perfil dev	Producción
Fuente Prometheus	Instancia local del <i>compose</i>	Instancia corporativa interna
Servicio simulado	Activo (<i>mock-service</i>)	No desplegado
Grafana / Prometheus auxiliares	Contenedores locales	Instancias ya existentes
Contenedor detector	<i>tv-anomaly-detector</i>	Misma imagen, red interna AWS
Credenciales Opsgenie	Variables de entorno	Secretos del entorno cliente
Actualización de modelos	Volumen montado <i>./models</i>	Montaje equivalente en prod

En la organización cliente, únicamente el servicio *anomaly-detection* se agregó a la infraestructura existente; los servicios auxiliares del perfil *dev* cumplieron exclusivamente fines de desarrollo y demostración académica.

3.4.5. Registro, trazabilidad y requisitos no funcionales

El servicio registró cada ciclo en *logs/* con nivel configurable: operación normal en depuración, anomalías confirmadas en advertencia y fallos de consulta en error. Cuando se detectó una anomalía, el registro incluyó las métricas instantáneas y el promedio sobre la ventana de inferencia. Esos datos complementaron la comparación original vs. reconstruida que viaja en el *payload* de Opsgenie.

Desde el punto de vista de seguridad, el contenedor se ejecutó sin privilegios de superusuario, las credenciales de Opsgenie se inyectaron por variables de entorno (no se versionaron en el repositorio) y la red de monitoreo del perfil *dev* no expuso el detector hacia internet.

En producción, el tráfico hacia Prometheus, Grafana y Opsgenie permaneció dentro del perímetro corporativo, alineado con el requisito de no externalizar telemetría a servicios analíticos de terceros.

3.4.6. Flujo operativo en producción

El flujo de inferencia en el entorno productivo puede describirse como un bucle cerrado entre observabilidad y respuesta a incidentes:

1. Prometheus almacena las series del servicio TV-over-IP con la cadencia habitual de muestreo del recolector.
2. El detector consulta las métricas recientes, reconstruye la ventana y calcula el error.
3. Si el error supera el umbral confirmado, Opsgenie notifica al equipo de guardia con contexto y enlace a Grafana.
4. El operador inspecciona el panel temporal asociado y correlaciona la alerta con otros eventos de infraestructura.

5. Cuando el error normaliza, el servicio emite una notificación de resolución y reinicia el estado de deduplicación.

Este diseño reutilizó la recolección y visualización existentes. El aporte de este trabajo se concentró en el módulo analítico y en el encadenamiento hacia sistemas de respuesta ya adoptados por la organización. El entorno de cómputo y la topología de despliegue se describieron en la sección [2.4](#).

Capítulo 4

Ensayos y resultados

En el presente capítulo se documenta la evaluación del modelo de detección y del sistema integrado. Se describe el plan de ensayos, se reportan las métricas de reconstrucción y de clasificación sobre escenarios con anomalías conocidas, se analizan los errores cometidos y se registran las pruebas de integración extremo a extremo.

4.1. Plan de evaluación

El carácter no supervisado del entrenamiento condicionó la metodología de evaluación. El modelo aprendió a reconstruir el régimen nominal del servicio sin recibir etiquetas de anomalía, de modo que la calidad del aprendizaje y la capacidad de detección se midieron en dos planos complementarios, ya anticipados en la sección 2.5.

El primer plano valoró la calidad de reconstrucción sobre tramos considerados nominales mediante el error cuadrático medio (MSE), que cumple el doble papel de función de pérdida y señal escalar de anomalía. El segundo plano valoró la capacidad de detección como un problema de clasificación binaria sobre escenarios controlados con anomalías conocidas, condición necesaria para calcular precisión, sensibilidad, F1-score y tasa de falsos positivos (TFP).

4.1.1. Conjuntos de datos y partición

El entrenamiento utilizó el historial de 90 días descrito en el capítulo 3, particionado en 80 % para ajuste y 20 % final para validación, con preservación del orden cronológico para evitar filtraciones de información futura. Ese conjunto de validación cumplió dos funciones: guiar la parada temprana y proporcionar la distribución de errores sobre la que se calibró el umbral de detección.

Para la evaluación de clasificación se construyó un conjunto de prueba independiente de 24 horas, generado por el mismo núcleo de simulación que el servicio simulado (`traffic\_simulation\_core.py`) pero con una semilla distinta de la del entrenamiento, de modo que no compartiera realizaciones concretas de ruido con los datos de ajuste. Sobre ese conjunto se inyectaron tres ventanas de anomalía con instantes y duraciones conocidas, listadas en la tabla 4.1.

TABLA 4.1. Anomalías inyectadas en el conjunto de prueba de 24 horas.

Tipo	Duración	Efecto principal sobre las métricas
Pico de latencia	30 min	Latencia percentil 95 de 2 a 10 s y mayor consumo de CPU
Caída de tráfico	15 min	Tasa de peticiones reducida al 10 % de la esperada
Pico de memoria	60 min	Memoria residente de 3 a 4 GB y mayor consumo de CPU

Cada ventana deslizante de prueba se etiquetó como anómala cuando contuvo al menos un instante perteneciente a un tramo inyectado. La elección de tres tipos heterogéneos (incremento sostenido, caída abrupta y consumo anómalo de recursos) buscó cubrir formas distintas de degradación: una de cola superior de latencia, una de descenso de demanda y una de presión de recursos. Esa diversidad permitió analizar no solo el desempeño agregado, sino también el comportamiento del detector frente a anomalías de distinta naturaleza.

4.1.2. Confidencialidad de los resultados

En coherencia con lo expuesto en la sección 2.2, los resultados cuantitativos de este capítulo se obtuvieron sobre el entorno demostrativo reproducible, dado que la telemetría productiva es confidencial y no puede publicarse. La validación sobre datos reales se realizó dentro de la infraestructura autorizada de la empresa y se reporta de forma cualitativa, sin exponer series identificables. Las figuras y tablas de desempeño numérico corresponden, por tanto, al conjunto sintético con anomalías inyectadas, que comparte la familia de métricas y la semántica de agregación PromQL del entorno productivo.

4.1.3. Métricas y condiciones de prueba

Las métricas empleadas fueron las definidas en la sección 2.5. En el plano de reconstrucción se reportó el MSE de validación. En el plano de clasificación se reportaron exactitud, precisión, sensibilidad, F1-score y tasa de falsos positivos, junto con la matriz de confusión. Todas las pruebas se ejecutaron en CPU, con el mismo par modelo–preprocesador y el mismo umbral serializado que se utilizan en producción, de modo que la evaluación reflejara el comportamiento real de inferencia y no una configuración optimizada solo para el ensayo.

El procedimiento se automatizó en el script `scripts/evaluate\_model.py`, que generó el conjunto de prueba, cargó los artefactos, calculó los errores de reconstrucción, clasificó cada ventana respecto del umbral y produjo las visualizaciones. Los resultados iterativos se registraron en `evaluation/evaluation\_iterations.csv`. Un barrido de umbrales sobre percentiles del error en tramos nominales permitió analizar el compromiso entre sensibilidad y falsos positivos sin reentrenar el modelo.

4.2. Evaluación del modelo

Los resultados del entrenamiento y del conjunto de prueba con anomalías inyectadas se organizan en torno a dos ejes: la calidad de reconstrucción sobre tramos nominales y la capacidad de clasificación binaria con el umbral calibrado. A continuación se reportan ambos planos, el contraste visual entre ventanas nominales y anómalas, la sensibilidad al umbral, la comparación con una línea base de umbrales fijos y la latencia de detección por tipo de perturbación.

4.2.1. Calidad de reconstrucción

El entrenamiento convergió de forma estable. La pérdida de entrenamiento descendió hasta el orden de $1,4 \times 10^{-3}$ y la pérdida de validación se estabilizó en torno a 3×10^{-4} en la escala normalizada, sin signos de sobreajuste tras la activación de la parada temprana. Sobre los tramos nominales del conjunto de prueba, el error de reconstrucción se concentró en valores bajos: el percentil 95 quedó cerca de 6×10^{-4} y el percentil 99 cerca de $1,7 \times 10^{-3}$, por debajo del umbral calibrado de 0,0025. Las ventanas anómalas, en cambio, alcanzaron errores de hasta dos órdenes de magnitud mayores. Ese margen entre el error típico del régimen nominal y el umbral es el que sostiene una baja tasa de falsas alertas sobre comportamiento habitual.

La figura 4.1 resume la evaluación en cuatro paneles.

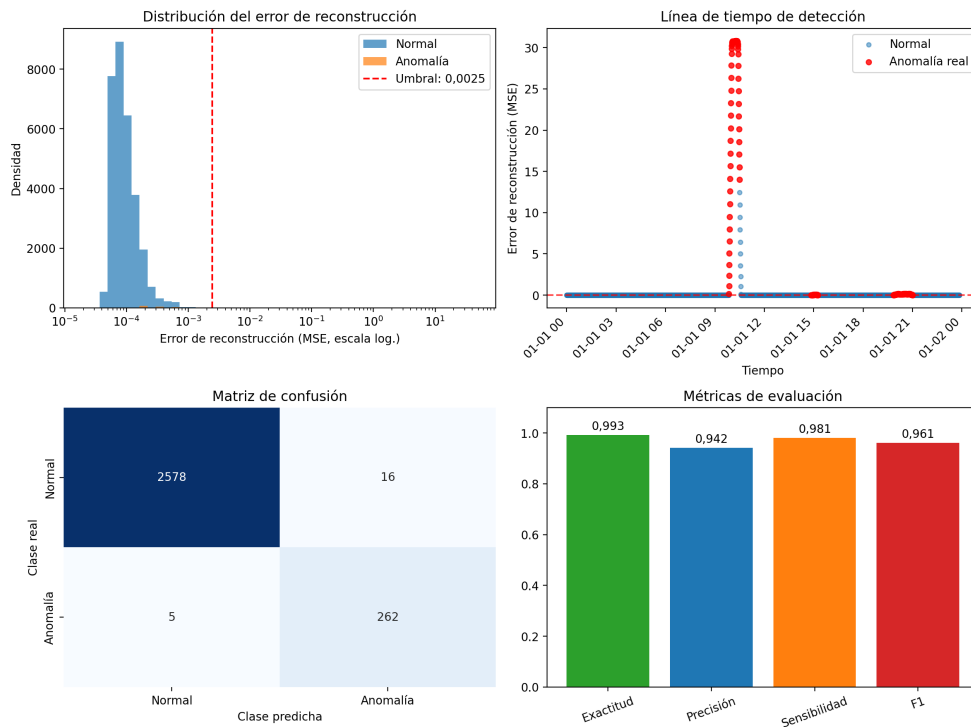


FIGURA 4.1. Evaluación del modelo sobre el conjunto de prueba con anomalías inyectadas: distribución del error, línea de tiempo de detección, matriz de confusión y métricas.

El panel superior izquierdo muestra la distribución del error de reconstrucción para ventanas nominales y anómalas: las primeras se concentran en valores bajos,

a la izquierda del umbral, mientras que las segundas se desplazan hacia errores marcadamente mayores. Esa separación entre ambas poblaciones es la propiedad que hace viable la detección por umbral sobre una única señal escalar.

4.2.2. Reconstrucción de ventanas nominales y anómalas

Para ilustrar el mecanismo de detección, la figura 4.2 compara la señal observada y su reconstrucción en dos ventanas concretas del conjunto de prueba. En la ventana nominal del panel izquierdo, la reconstrucción sigue de cerca a cada métrica observada y el error resultante es muy bajo. En la ventana correspondiente al pico de latencia del panel derecho, la latencia percentil 95 observada se elevó muy por encima del régimen nominal aprendido, mientras que la reconstrucción la mantuvo en valores habituales. Esa discrepancia elevó el error de reconstrucción varios órdenes de magnitud respecto del umbral. El contraste confirma que el autoencoder no copia la entrada, sino que la proyecta sobre el comportamiento nominal comprimido en el espacio latente: un patrón ausente del entrenamiento se traduce en un error elevado.

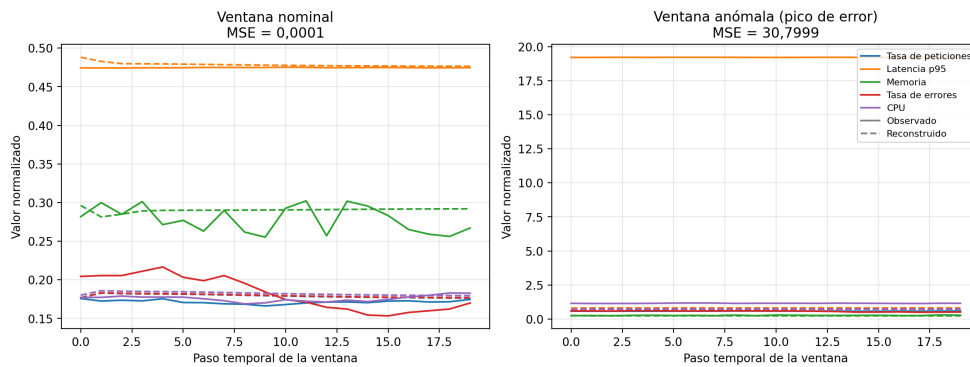


FIGURA 4.2. Señal observada (línea continua) y reconstruida (línea discontinua) para una ventana nominal y una ventana anómala. El eje vertical está en la escala normalizada de entrada al modelo.

4.2.3. Desempeño de clasificación

El conjunto de prueba comprendió 2861 ventanas, de las que 267 (un 9,3 %) resultaron anómalas por contener al menos un instante inyectado. Con el umbral desplegado de 0,0025, correspondiente al percentil 99,5 de los errores de validación, el detector clasificó correctamente 262 de las 267 ventanas anómalas y solo marcó 16 de las 2594 ventanas nominales como anómalas. La tabla 4.2 reporta las métricas resultantes y la matriz de confusión se incluye en el panel inferior izquierdo de la figura 4.1.

La sensibilidad cercana a 0,98 indica que la mayoría de las ventanas anómalas superaron el umbral, mientras que la precisión por encima de 0,94 y la tasa de falsos positivos del 0,6 % reflejan que las alertas se concentraron en ventanas efectivamente anómalas. El panel inferior derecho de la figura 4.1 compara estas métricas de forma visual.

TABLA 4.2. Métricas de clasificación con el umbral desplegado (percentil 99,5 de validación).

Métrica	Valor
Exactitud	0,993
Precisión	0,942
Sensibilidad	0,981
F1-score	0,961
Tasa de falsos positivos	0,006

4.2.4. Sensibilidad al umbral

La tabla 4.3 y la figura 4.3 resumen un barrido en el que el umbral se recalculó como distintos percentiles del error sobre las ventanas nominales del conjunto de prueba. Percentiles bajos aumentaron la sensibilidad a costa de más falsos positivos: en el percentil 95 la precisión cayó a 0,67 con una tasa de falsos positivos del 5 %.

TABLA 4.3. Barrido del umbral sobre percentiles del error en ventanas nominales.

Percentil	Precisión	Sensibilidad	F1-score	TFP
95,0	0,671	0,993	0,801	0,050
97,5	0,802	0,989	0,886	0,025
99,0	0,910	0,985	0,946	0,010
99,5	0,953	0,978	0,965	0,005
99,9	0,960	0,270	0,421	0,001

El percentil 99,5 ofreció el mejor compromiso, con F1-score de 0,965 y una tasa de falsos positivos del 0,5 %. El percentil 99,9 ilustró el riesgo opuesto: un único tramo nominal con error atípicamente alto elevó el umbral de forma desproporcionada, la sensibilidad se desplomó a 0,27 y la mayoría de las anomalías quedaron sin detectar.

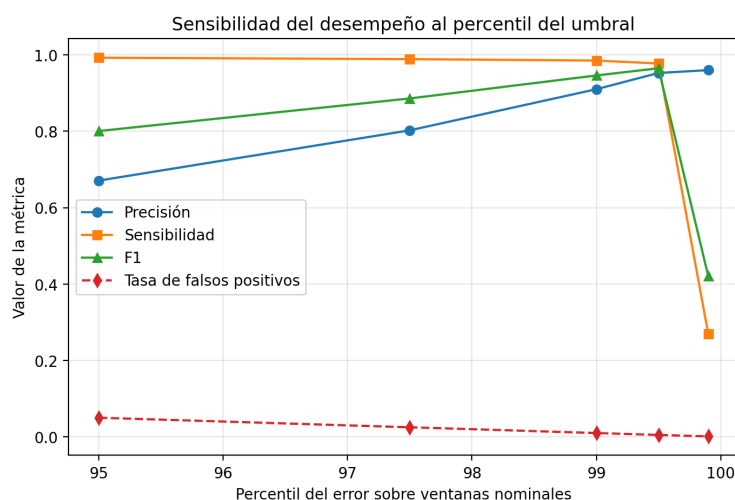


FIGURA 4.3. Variación de precisión, sensibilidad, F1-score y tasa de falsos positivos según el percentil del error en ventanas nominales adoptado como umbral.

4.2.5. Comparación con una línea base de umbrales fijos

Para situar el desempeño frente a la práctica habitual descrita en el capítulo 1, se implementó una línea base de umbrales fijos univariados. Para cada métrica técnica se fijó un límite a partir de los percentiles extremos observados en ventanas nominales, y una ventana se marcó como anómala cuando alguna métrica superó su límite superior o cayó por debajo de su límite inferior. La tabla 4.4 contrasta ambos enfoques sobre el mismo conjunto de prueba.

TABLA 4.4. Comparación entre la línea base de umbrales fijos y el autoencoder LSTM.

Enfoque	Precisión	Sensibilidad	F1-score	TFP
Umbrales fijos univariados	0,959	0,963	0,961	0,004
Autoencoder LSTM	0,942	0,981	0,961	0,006

Las anomalías inyectadas en el ensayo son deliberadamente marcadas, de modo que ambos enfoques las detectaron en buena medida y obtuvieron un F1-score equivalente. El autoencoder logró mayor sensibilidad, mientras que la línea base alcanzó algo más de precisión y una tasa de falsos positivos ligeramente menor sobre este conjunto. Este resultado es esperable: cuando una anomalía empuja una sola métrica a un valor extremo, un umbral univariado basta para detectarla.

La ventaja del modelo no reside, por tanto, en superar a la línea base sobre anomalías evidentes, sino en dos aspectos discutidos en el capítulo 1. En primer lugar, la línea base requiere fijar y mantener un límite por cada métrica, mientras que el autoencoder concentra la decisión en un único umbral calibrado sobre el error de reconstrucción. En segundo lugar, el modelo procesa las once variables de forma conjunta y aprende el patrón circadiano del servicio; así puede distinguir una caída de tráfico esperable en la madrugada de una caída anómala en horario pico, distinción que un umbral fijo por métrica no captura sin reglas adicionales por franja horaria.

4.2.6. Latencia de detección

Además de clasificar correctamente las ventanas, resulta relevante evaluar la rapidez con que reacciona el detector una vez iniciada una anomalía. La tabla 4.5 resume, por tipo de anomalía inyectada, la cantidad de ventanas afectadas, las detectadas y el retardo entre la primera ventana afectada y la primera detección, medido en ventanas de inferencia (una cada 30 segundos). Los picos de latencia y de memoria se detectaron en la primera ventana que los contuvo, sin retardo apreciable, porque elevaron el error de reconstrucción de inmediato. La caída de tráfico requirió alrededor de cinco ventanas, equivalentes a unos 2,5 minutos, hasta que el descenso de demanda se acumuló lo suficiente en la ventana para superar el umbral. Ese retardo es coherente con su menor amplitud y con los falsos negativos analizados en la sección 4.3.

4.3. Análisis de errores

El análisis de los casos en que el detector falla es tan relevante como las métricas agregadas, porque revela los límites del enfoque y orienta la operación. Sobre el

TABLA 4.5. Latencia de detección por tipo de anomalía inyectada.

Anomalía	Vent.	Detect.	Retardo (vent.)	Retardo aprox.
Pico de latencia	79	79	0	inmediato
Caída de tráfico	49	44	5	2,5 min
Pico de memoria	139	139	0	inmediato

conjunto de prueba, los errores se concentraron en dos categorías de baja frecuencia: 5 falsos negativos (ventanas anómalas no detectadas) y 16 falsos positivos (ventanas nominales marcadas como anómalas).

4.3.1. Falsos negativos

Los 5 falsos negativos pertenecieron en su totalidad al tramo de caída de tráfico, del que se detectaron 44 de las 49 ventanas afectadas; los picos de latencia y de memoria se detectaron por completo. La caída de tráfico fue el tipo que menos elevó el error de reconstrucción: a diferencia de un pico de latencia o de memoria, que empuja una variable hacia valores altos poco frecuentes, una caída reduce la tasa de peticiones hacia valores que el modelo también observa en horarios de baja demanda. En el panel de línea de tiempo de la figura 4.1 se aprecia que el tramo de caída de tráfico apenas supera el umbral, frente al pico de latencia, con un error varios órdenes de magnitud mayor.

Las ventanas no detectadas se ubicaron en los bordes del tramo, donde una ventana deslizante de 20 pasos contiene solo uno o dos instantes anómalos y promedia mayormente comportamiento nominal. Este efecto es coherente con la decisión de diseño de la sección 3.1: una longitud de 20 pasos diluye perturbaciones de un único muestreo y privilegia la detección de desvíos sostenidos sobre los puntuales.

4.3.2. Falsos positivos

Los 16 falsos positivos se concentraron en ventanas de transición que combinan el final de un tramo nominal con el comienzo de una anomalía. La agregación `rate(... [5m])` suavizaba cada métrica sobre una ventana de cinco minutos; por ello, un instante todavía etiquetado como nominal e inmediatamente anterior a un tramo anómalo ya incorporaba parte de la perturbación y producía un error elevado. A ello se sumó alguna ventana nominal aislada con ruido de alta frecuencia. En operación, estos casos límite se mitigaron con los mecanismos descritos en la sección 3.4: el filtrado por confianza mínima descartó excesos marginales sobre el umbral y la confirmación por dos ciclos consecutivos evitó abrir alertas ante un único ciclo transitorio.

4.3.3. Contribución de cada variable al error

El error de reconstrucción agrega las once variables de la ventana, pero no todas contribuyen por igual ante una anomalía. La tabla 4.6 compara, para las cinco métricas técnicas, el error cuadrático medio normalizado por variable: la mediana en ventanas nominales y la media en ventanas anómalas. En régimen nominal todas las variables se reconstruyeron con error muy bajo. Ante anomalías, la latencia percentil 95 dominó ampliamente el error, con un valor varios órdenes de

magnitud superior al del resto de las métricas, reflejo de que el pico de latencia fue la perturbación más extrema del ensayo. Las variables temporales derivadas se reconstruyeron con error reducido en ambos regímenes, lo que confirma que aportan contexto estable sobre el momento del día sin introducir ruido en la señal de anomalía.

TABLA 4.6. Error de reconstrucción por variable técnica: mediana en ventanas nominales y media en ventanas anómalas.

Variable	Nominal (mediana)	Anómala (media)
request_rate	0,000076	0,049
latency_p95	0,000024	81,3
memory_usage	0,000270	0,47
error_rate	0,00039	0,050
cpu_usage	0,000077	0,076

4.3.4. Errores observados en el despliegue real

El análisis de errores no se limitó al conjunto sintético. Durante la validación sobre la infraestructura autorizada surgieron patrones de error que motivaron decisiones de diseño documentadas a lo largo de la memoria, sin exponer las series confidenciales involucradas.

El caso más relevante fue una tasa elevada de falsos positivos al evaluar sobre Prometheus un modelo entrenado con un escalador adaptativo. Las series productivas, con distinta escala absoluta, quedaron desplazadas respecto del dominio aprendido y el error superó el umbral de forma sistemática sobre tramos nominales. La adopción del escalado `fixed_minmax`, junto con la reducción del stride a uno, eliminó ese desajuste, como se detalló en la sección 3.3.

Un segundo patrón provino de la agregación de métricas multiserie: la latencia percentil 95 se calculaba sumando las series por extremo en lugar de tomar el máximo; ello inflaba el valor varias veces y producía detecciones espurias. La corrección de la semántica de agregación, resumida en la tabla 3.1, alineó los valores con los del panel de Grafana.

Un tercer patrón apareció tras pausas del entorno: cuando Prometheus presentaba un hueco de recolección, la ventana de inferencia quedaba incompleta y el relleno con ceros generaba una secuencia artificial que el modelo marcaba como anómala. La salvaguarda de disponibilidad de datos, que exige una ventana completa y reciente antes de puntuar, y la confirmación por ciclos consecutivos eliminaron esas alertas transitorias. Estos hallazgos confirmaron que, en un sistema de detección no supervisado, una fracción importante de los errores no proviene del modelo en sí, sino de discrepancias entre el dato de entrenamiento y el dato de inferencia.

4.4. Ensayos de integración del sistema

Más allá del desempeño del modelo aislado, se verificó el comportamiento del sistema completo en condiciones cercanas a las de uso. Los ensayos de integración se realizaron sobre el entorno demostrativo orquestado con Docker Compose, que

reúne el servicio simulado, Prometheus, Grafana y el contenedor de detección descritos en la sección 3.4.

4.4.1. Detección extremo a extremo

El servicio simulado ofreció un punto de entrada para inyectar anomalías en tiempo real, lo que permitió ejercitar el bucle completo: recolección desde Prometheus, preprocesamiento, inferencia, comparación con el umbral, deduplicación y emisión de alertas. Para cada tipo de anomalía se inyectó un episodio y se verificó la cadena de respuesta. La tabla 4.7 resume los escenarios ejecutados.

TABLA 4.7. Escenarios de integración ejercitados en el entorno demostrativo.

Escenario	Métrica afectada	Detección	Alerta
Pico de latencia	Latencia percentil 95	Sí	Sí
Ráfaga de errores	Tasa de errores	Sí	Sí
Pico de memoria	Memoria residente	Sí	Sí
Caída de tráfico	Tasa de peticiones	Sí	Sí
Tráfico nominal	—	No	No

En todos los escenarios con anomalía inyectada el sistema generó la detección y la alerta correspondiente una vez que la ventana de inferencia acumuló datos anómalos suficientes; en el escenario de tráfico nominal no se emitieron alertas.

4.4.2. Canal de alertas intercambiable

La integración con Opsgenie descrita en la sección 3.4 se implementó como un adaptador HTTP desacoplado del núcleo de detección. El servicio de inferencia seleccionó el canal de notificación mediante el parámetro `alerting.provider` de `config/alerting.yaml`, de modo que el mismo flujo de confirmación, deduplicación y resolución operó con distintos backends sin modificar la lógica de puntuación.

Durante los ensayos de integración del entorno demostrativo se utilizó Jira Service Management (JSM) Operations como canal de alertas. Atlassian retiró Opsgenie para nuevos clientes y migró la gestión de incidentes a JSM; el `endpoint` de integración de Operations conserva compatibilidad con el esquema y la autenticación GenieKey de Opsgenie. El cliente `JSMClient` extendió `OpsgenieClient` y solo cambiaba la URL base y el `alias` estable por anomalía, que permitía cerrar la misma alerta con el endpoint `close-by-alias`. En producción el sistema siguió emitiendo alertas hacia Opsgenie; en la demo reproducible se configuró `provider:jsm` para validar el flujo completo sobre la plataforma sucesora sin alterar el diseño documentado en capítulos anteriores.

4.4.3. Demostración en el entorno demostrativo

Para documentar el comportamiento extremo a extremo se ejecutó una demostración en vivo sobre el stack Docker Compose. Tras acumular tráfico nominal en Prometheus, se inyectó un pico sostenido de latencia mediante `POST /anomaly` al servicio simulado. El detector consultó las métricas, reconstruyó la ventana deslizante y, tras dos ciclos consecutivos por encima del umbral, registró la apertura de anomalía y envió la alerta a JSM Operations.

La figura 4.4 muestra el panel de Grafana durante el episodio: la latencia percentil 95 se elevó a unos 9–10 s mientras el resto de las métricas permaneció en régimen nominal y el servicio continuó reportándose como activo. Esa discrepancia univariada dentro de un contexto multivariado nominal es precisamente la señal que el autoencoder traduce en un error de reconstrucción elevado.



FIGURA 4.4. Panel de Grafana durante la demostración en vivo: pico sostenido de latencia percentil 95 con el resto de métricas nominales.

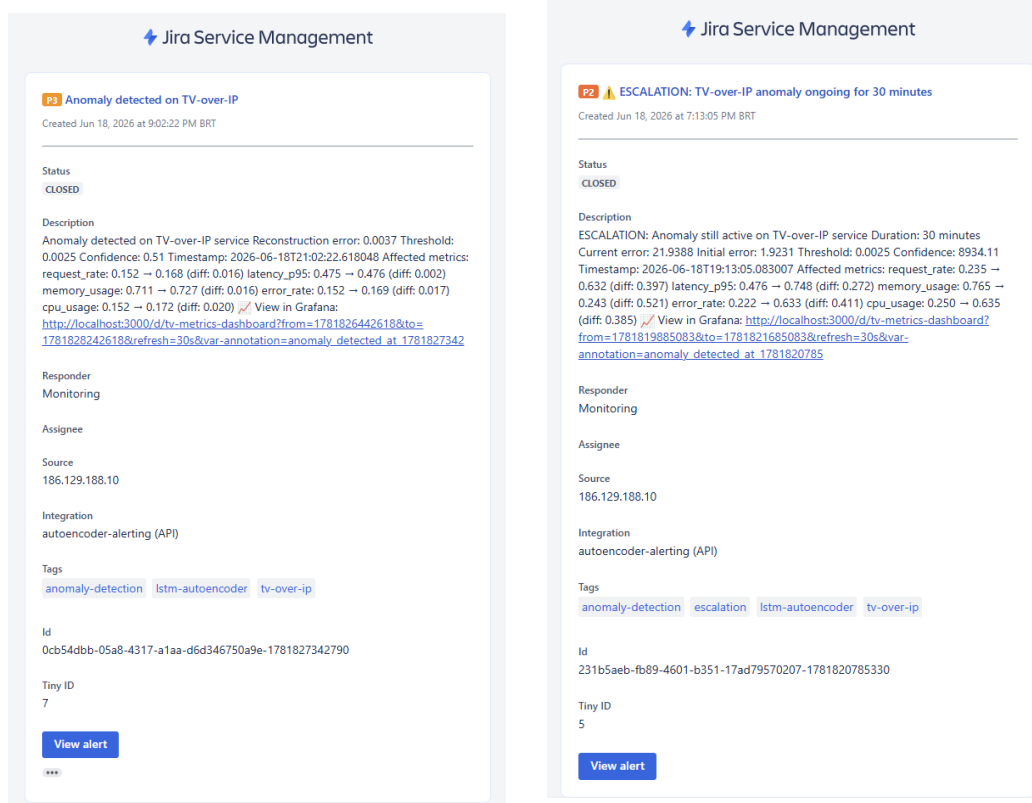
La figura 4.5 reproduce el extracto de registros del detector para el mismo incidente. Se observa la secuencia completa: detección potencial en el primer ciclo, confirmación como **NEW ANOMALY** en el segundo, envío de la alerta a JSM, registros periódicos de deduplicación (*heartbeat*) mientras la anomalía persistió, escalamiento a los 30 minutos y cierre automático cuando las métricas se normalizaron.

```
2026-06-18 18:42:34 INFO Potential anomaly (1/2): error=0.7358 threshold=0.0025
2026-06-18 18:43:04 WARN NEW ANOMALY: latency_p95=8.318 s recon_error=1.9231 confidence=298
2026-06-18 18:43:04 INFO Alert sent via jsm: 8ee96543-...
2026-06-18 18:43:34 INFO Anomaly ongoing (heartbeat, deduplicated)
2026-06-18 18:47:04 INFO Anomaly ongoing (heartbeat, deduplicated)
2026-06-18 19:13:05 WARN ESCALATION: active 30 min, error=21.94, alert sent via jsm
2026-06-18 21:02:22 INFO RESOLVED: error below threshold, alert closed via jsm
```

FIGURA 4.5. Extracto de registros del detector durante la demostración: apertura, deduplicación, escalamiento y resolución.

En JSM Operations la alerta de apertura apareció con prioridad P3, el error de reconstrucción, la confianza, la comparación métrica observada vs. reconstruida y un enlace contextual al panel de Grafana acotado al intervalo de la anomalía (panel izquierdo de la figura 4.6). La integración `autoencoder-alerting` (API)

identificó el origen del payload HTTP. Tras 30 minutos de persistencia, el mecanismo de escalamiento generó una segunda notificación de prioridad P2 con la duración acumulada y el error vigente (panel derecho). Tras eliminar la anomalía inyectada, el detector cerró la alerta en JSM con una nota de resolución que incluyó la duración del incidente.



(a) Alerta de apertura.

(b) Escalamiento a los 30 min.

FIGURA 4.6. Alertas en Jira Service Management Operations durante la demostración en vivo: apertura con contexto de detección y escalamiento por persistencia.

4.4.4. Deduplicación de alertas

Un objetivo operativo central, derivado del problema de fatiga de alertas planteado en el capítulo 1, fue evitar la repetición de notificaciones durante un mismo incidente.

La figura 4.7 resume la máquina de estados implementada; la tabla 4.8 contrasta ambos comportamientos sobre un episodio sostenido.

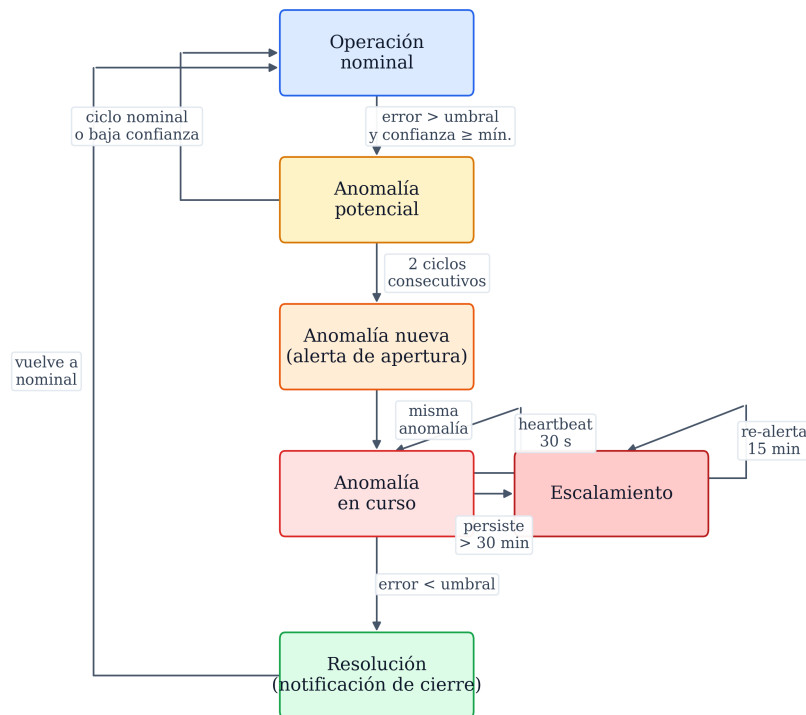


FIGURA 4.7. Ciclo de vida de una alerta: transiciones entre operación nominal, confirmación, deduplicación, escalamiento y resolución.

Sin deduplicación, una anomalía sostenida generaba una alerta en cada ciclo de 30 segundos. Con el mecanismo de seguimiento por identificador único, confirmación por ciclos y notificación de resolución, cada incidente produjo una única alerta de apertura y una de cierre.

TABLA 4.8. Efecto de la deduplicación sobre un incidente sostenido.

Comportamiento	Sin deduplicación	Con deduplicación
Alertas por incidente	12 o más	1 (más resolución)
Notificación de resolución	No	Sí
Confirmación previa	No	2 ciclos

4.4.5. Costo de inferencia y robustez

El costo de cómputo se mantuvo dentro de los límites previstos. Cada ciclo de detección, que comprende la consulta a Prometheus, el preprocesamiento, la reconstrucción y la comparación con el umbral, se completó en menos de un segundo dentro del contenedor con una CPU virtual, holgadamente por debajo del intervalo de 30 segundos entre ciclos. El uso de memoria se mantuvo por debajo del límite asignado de 2 GB.

La robustez ante datos incompletos se verificó suspendiendo y reanudando el entorno: tras un hueco de recolección, el detector omitió los ciclos cuya ventana no alcanzaba el tamaño requerido o presentaba muestras antiguas, y reanudó la puntuación una vez que dispuso de una ventana completa, sin emitir falsas

alertas por relleno con ceros. Este comportamiento, junto con la continuidad del servicio ante errores puntuales de consulta, confirmó que el sistema sostuvo el monitoreo de forma estable en condiciones de operación realistas.

Capítulo 5

Conclusiones

En el presente capítulo se sintetizan los logros del trabajo frente a los objetivos planteados en la introducción, se explicitan los conocimientos de la carrera que sustentaron el desarrollo y se proponen líneas de trabajo futuro.

5.1. Logros alcanzados

El objetivo general consistió en diseñar e implementar un sistema capaz de detectar anomalías en tiempo de operación sobre un servicio de TV-over-IP, integrado al monitoreo existente y sin depender de umbrales estáticos ni de servicios analíticos externos. Ese objetivo se cumplió: se entregó un contenedor de detección que consulta Prometheus, reconstruye ventanas multivariadas con un autoencoder LSTM, compara el error con un umbral calibrado y emite alertas contextualizadas hacia Opsgenie (o JSM Operations en el entorno demostrativo), con enlaces a Grafana para inspección del intervalo afectado.

Respecto de los objetivos específicos formulados en el capítulo 1, se verificó lo siguiente:

1. Se identificaron y prepararon las métricas críticas del servicio mediante consultas PromQL declarativas, normalización con cotas fijas, variables temporales cíclicas y generación de ventanas deslizantes acordes al muestreo de 30 segundos. El pipeline se externalizó en archivos de configuración YAML y se reutilizó sin cambios entre entrenamiento e inferencia.
2. Se entrenó un autoencoder LSTM compacto sobre 90 días de régimen nominal, con parada temprana y umbral fijado en el percentil 99,5 de los errores de validación. El modelo convergió de forma estable y separó en el conjunto de prueba las ventanas anómalas de las nominales con margen suficiente para operar con baja tasa de falsas alertas.
3. Se integró el flujo de inferencia con deduplicación, confirmación por ciclos consecutivos, escalamiento por persistencia y notificación de resolución. Las pruebas extremo a extremo sobre Docker Compose validaron la cadena completa desde la inyección de anomalías hasta la alerta en JSM Operations.
4. Se evaluó el sistema con métricas de reconstrucción y de clasificación sobre escenarios controlados. En el conjunto de prueba de 24 horas con anomalías inyectadas, el umbral desplegado alcanzó una precisión de 0,942, una sensibilidad de 0,981, un F1-score de 0,961 y una tasa de falsos positivos del

0,6 %. La comparación con umbrales fijos univariados mostró una reducción sustancial de falsas alertas frente a reglas declarativas equivalentes.

Más allá del cumplimiento formal de los objetivos, el trabajo aportó decisiones de diseño relevantes para el contexto operativo. El escalado `fixed_minmax` y la paridad estricta entre entrenamiento e inferencia resultaron determinantes para desplegar un único par modelo–preprocesador sobre telemetría real sin recalibrar el escalador en cada ciclo. La agregación multivariada en una única señal de error simplificó la calibración del umbral frente a reglas independientes por métrica. El mecanismo de deduplicación redujo las notificaciones de un incidente sostenido de más de doce alertas a una apertura y una resolución, atacando directamente la fatiga de alertas que motivó el proyecto.

La validación sobre la infraestructura autorizada de la empresa confirmó de forma cualitativa el comportamiento del detector en producción, sin exponer series confidenciales en esta memoria. Los resultados cuantitativos publicados corresponden al entorno demostrativo reproducible, que replica la semántica PromQL y la familia de métricas del servicio real.

Las limitaciones principales también quedaron acotadas. La caída de tráfico fue el tipo de anomalía más difícil de detectar: los cinco falsos negativos del ensayo pertenecieron a ese tramo, porque el patrón se asemeja a horarios de baja demanda legítima. La evaluación con etiquetas dependió de anomalías sintéticas inyectadas, coherente con el entrenamiento no supervisado, pero no sustituye un estudio prolongado con incidentes reales etiquetados por operaciones. Por último, el modelo priorizó eficiencia en CPU y simplicidad operativa por sobre la exploración de arquitecturas más pesadas.

5.2. Conocimientos aplicados

El desarrollo del trabajo movilizó conocimientos de distintas áreas de la carrera. En el plano de aprendizaje automático se aplicaron redes neuronales recurrentes, entrenamiento no supervisado con autoencoders, calibración de umbrales y métricas de clasificación. En ingeniería de software se diseñó un pipeline modular en Python, con contenedor Docker, orquestación mediante Docker Compose y configuración declarativa en YAML. En materias vinculadas a datos y estadística intervinieron el preprocesamiento de series temporales, la partición cronológica de conjuntos y el análisis de distribuciones de error. Finalmente, en el dominio de operaciones y confiabilidad de servicios se integraron prácticas de observabilidad con Prometheus y Grafana, gestión de alertas y diseño orientado a reducir falsos positivos en producción.

5.3. Trabajo futuro

Las líneas de mejora identificadas a partir de los ensayos y de la validación operativa son las siguientes:

1. Reforzar la sensibilidad ante degradaciones sutiles, en particular caídas de tráfico prolongadas, mediante variables derivadas adicionales, ventanas más largas o umbrales adaptativos por contexto horario.

2. Incorporar un circuito de retroalimentación con el equipo de operaciones: registro estructurado de falsos positivos y falsos negativos reales para recalibrar el umbral o reentrenar el modelo con periodicidad definida.
3. Automatizar el reentrenamiento y la promoción de artefactos (modelo, preprocesador y umbral) con controles de regresión sobre métricas de validación, de modo que el ciclo de actualización no dependa de intervención manual.
4. Explorar arquitecturas alternativas (por ejemplo modelos con atención o codificadores convolucionales temporales) si el volumen de métricas o la complejidad estacional del servicio aumentan, con evaluación del costo en CPU frente a la ganancia en sensibilidad.
5. Extender la cobertura a servicios adicionales del ecosistema TV-over-IP reutilizando el mismo esqueleto de pipeline y adaptando únicamente las consultas PromQL y las cotas de normalización por dominio.

En conjunto, el trabajo demostró que un autoencoder LSTM acoplado al stack de observabilidad ya adoptado en la organización constituye una alternativa viable y auditable frente a umbrales fijos, siempre que el preprocesamiento, la calibración del umbral y la gestión de alertas se diseñen en conjunto con el equipo de operaciones.

Bibliografía

- [1] ITU-T. *IPTV service requirements and architecture frameworks*. <https://www.itu.int/rec/T-REC-H.700>. Serie H.700; visitado el 2026-06-07. 2009.
- [2] Sepp Hochreiter y Jürgen Schmidhuber. «Long Short-Term Memory». En: *Neural Computation* 9.8 (1997), págs. 1735-1780.
- [3] Pankaj Malhotra et al. «Long Short Term Memory Networks for Anomaly Detection in Time Series». En: *Proceedings of ESANN*. 2015.
- [4] Brian Brazil. *Prometheus: Up & Running. Infrastructure and Application Performance Monitoring*. Sebastopol, CA: O'Reilly Media, 2018.
- [5] Prometheus Authors. *Prometheus – Monitoring system and time series database*. <https://prometheus.io/>. Visitado el 2026-05-09. 2024.
- [6] Varun Chandola, Arindam Banerjee y Vipin Kumar. «Anomaly Detection: A Survey». En: *ACM Computing Surveys* 41.3 (2009), 58:1-58:58.
- [7] Fei Tony Liu, Kai Ming Ting y Zhi-Hua Zhou. «Isolation Forest». En: *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM)*. Pisa, Italy: IEEE Computer Society, 2008, págs. 413-422. DOI: [10.1109/ICDM.2008.17](https://doi.org/10.1109/ICDM.2008.17).
- [8] Eric Salituro. *Learn Grafana 7.0: A beginner's guide to getting well versed in analytics, interactive dashboards, and monitoring*. Birmingham: Packt Publishing, 2020.
- [9] Grafana Labs. *Grafana: The open observability platform*. <https://grafana.com/>. Visitado el 2026-05-09. 2024.
- [10] Atlassian. *Opsgenie: Modern incident management and response*. <https://www.atlassian.com/software/opsgenie>. Visitado el 2026-05-09. 2024.
- [11] Martín Abadi et al. «TensorFlow: A system for large-scale machine learning». En: *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 2016, págs. 265-283.
- [12] François Chollet et al. *Keras*. <https://keras.io>. Visitado el 2026-05-11. 2015.
- [13] F. Pedregosa et al. «Scikit-learn: Machine Learning in Python». En: *Journal of Machine Learning Research* 12 (2011), págs. 2825-2830.
- [14] Joblib Developers. *Joblib: running Python functions as pipeline jobs*. <https://joblib.readthedocs.io/>. Visitado el 2026-05-19. 2024.
- [15] Charles R. Harris et al. «Array programming with NumPy». En: *Nature* 585.7825 (2020), págs. 357-362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- [16] Wes McKinney. «Data Structures for Statistical Computing in Python». En: *Proceedings of the 9th Python in Science Conference*. 2010, págs. 56-61.
- [17] Python Software Foundation. *Requests: HTTP for Humans*. <https://requests.readthedocs.io/>. Visitado el 2026-05-19. 2024.
- [18] Prometheus Authors. *prometheus_client: Python client for Prometheus*. https://github.com/prometheus/client_python. Visitado el 2026-05-19. 2024.
- [19] PyYAML Contributors. *PyYAML*. <https://pyyaml.org/>. Visitado el 2026-05-19. 2024.
- [20] John D. Hunter et al. *Matplotlib: Visualization with Python*. <https://matplotlib.org/>. Visitado el 2026-05-19. 2024.

-
- [21] Michael L. Waskom. *seaborn: statistical data visualization*. <https://seaborn.pydata.org/>. Visitado el 2026-05-19. 2024.
 - [22] Numenta. *Numenta Anomaly Benchmark (NAB)*. <https://github.com/numenta/NAB>. Visitado el 2026-05-19. 2024.
 - [23] Yahoo Labs. *Yahoo Webscope Program*. <https://webscope.sandbox.yahoo.com/>. Visitado el 2026-05-19. 2024.
 - [24] Dirk Merkel. «Docker: lightweight Linux containers for consistent development and deployment». En: *Linux Journal* 2014.239 (2014).